



# **Arba Minch University**

Arba Minch Institute of Technology

Faculty of Computing and Software Engineering

**Software Quality Assurance and Testing (SENG3164)**

Learners Material

By: Mikiyas B.

Arba Minch, Ethiopia

March, 2023

## Table of Contents

|                                                                           |           |
|---------------------------------------------------------------------------|-----------|
| List of Figures .....                                                     | iii       |
| List of Tables.....                                                       | iv        |
| Acronyms and Abbreviations.....                                           | v         |
| <b>Chapter One: Introduction.....</b>                                     | <b>1</b>  |
| 1.1 Software Quality .....                                                | 2         |
| 1.2 Quality Characteristics .....                                         | 3         |
| 1.3 Software Quality Activities.....                                      | 3         |
| 1.4 Software Quality Assurance.....                                       | 5         |
| 1.5 Software Quality Factor .....                                         | 6         |
| 1.5.1 Product operation software quality factors .....                    | 7         |
| 1.5.2 Product revision software quality factors.....                      | 9         |
| 1.5.3 Product transition software quality factors .....                   | 10        |
| 1.6 Software Quality Assurance Components.....                            | 11        |
| 1.6.1 Project Lifecycle Activities and Assessment .....                   | 14        |
| 1.6.2 Infrastructure components for error prevention and Improvement..... | 15        |
| 1.7 Software Quality Management.....                                      | 16        |
| <b>Chapter Two: Software Testing .....</b>                                | <b>19</b> |
| 2.1 Software Testing Terminology.....                                     | 20        |
| 2.2 Observation about Testing .....                                       | 22        |
| 2.3 Objective of Testing.....                                             | 22        |
| 2.4 Fundamental principles of testing .....                               | 24        |
| 2.5 Software Testing Life Cycle – Different Stages of Testing .....       | 24        |
| 2.6 Levels of Software of Testing .....                                   | 36        |
| 2.6.1 Unit Testing.....                                                   | 37        |
| 2.6.2 Integration Testing .....                                           | 41        |
| 2.6.3 System Testing.....                                                 | 42        |
| 2.6.4 Acceptance Testing .....                                            | 52        |
| <b>Chapter Three: Software Testing Techniques.....</b>                    | <b>55</b> |
| 3.1 Classification of software Testing Techniques .....                   | 56        |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| 3.1.1 White Box Strategies.....                                   | 56        |
| 3.1.2 Black Box Strategies .....                                  | 59        |
| 3.1.3 Grey Box Strategies .....                                   | 63        |
| <b>Chapter Four: Test Management.....</b>                         | <b>65</b> |
| 4.1 Test Management .....                                         | 66        |
| 4.2. Test Management Responsibilities.....                        | 66        |
| 4.3. Profile of Test Personnel .....                              | 66        |
| 4.4 Test Maturity Model.....                                      | 68        |
| 4.5 Test Plan.....                                                | 69        |
| 4.5.1 Test Plan Components.....                                   | 70        |
| <b>Chapter Five: Quality Management System and Standards.....</b> | <b>74</b> |
| 5.1 Quality Management System .....                               | 74        |
| 5.1.1 QMS: Plan- Do- Check- Act .....                             | 74        |
| 5.1.2 Quality Standards .....                                     | 75        |
| 5.1.3 Organizations for Standard .....                            | 76        |
| <b>Bibliography .....</b>                                         | <b>82</b> |

## List of Figures

|                                                    |    |
|----------------------------------------------------|----|
| Figure 1:McCall’s quality factor model tree .....  | 7  |
| Figure 2:SQA System Overview .....                 | 12 |
| Figure 3: Objective of software Testing .....      | 23 |
| Figure 4: Software Testing Life Cycle .....        | 24 |
| Figure 5: Test Estimation Process .....            | 27 |
| Figure 6: Stages of Test Closure.....              | 35 |
| Figure 7: Level of Software Testing.....           | 37 |
| Figure 8:Test Harness .....                        | 40 |
| Figure 9:Unit Test worksheet .....                 | 41 |
| Figure 10: CMM Level in Test Management .....      | 68 |
| Figure 11: DMAIC The Improvement Methodology ..... | 79 |

## List of Tables

|                                                                   |    |
|-------------------------------------------------------------------|----|
| Table 1: Task to Sub Tasks.....                                   | 29 |
| Table 2: Allocate each task to a team member .....                | 29 |
| Table 3: Assigning weightage to each Using functional point ..... | 31 |
| Table 4: Estimate Duration for the Task.....                      | 32 |
| Table 5: Level of Software Testing Conclusion .....               | 53 |
| Table 6: Decision Table Example.....                              | 63 |

## Acronyms and Abbreviations

|         |                                                   |
|---------|---------------------------------------------------|
| ❖ AJAX  | Asynchronous JavaScript and XML                   |
| ❖ AUT   | Application Under Test                            |
| ❖ CSV   | Comma Separated Values                            |
| ❖ DT    | Decision Tree                                     |
| ❖ GUI   | Graphic User Interface                            |
| ❖ IDE   | Integrated Development Environment                |
| ❖ IEEE  | Institute of Electrical and Electronics Engineers |
| ❖ LOC   | Line of Code                                      |
| ❖ MTTF  | Mean Time to Failure                              |
| ❖ MTTR  | Mean Time to Repair                               |
| ❖ QTP   | Quick Test Professional                           |
| ❖ QMS   | Quality Management System                         |
| ❖ SDLC  | Software Development Life Cycle                   |
| ❖ SQA   | Software Quality Assurance                        |
| ❖ SUT   | System Under Test                                 |
| ❖ UI    | User Interface                                    |
| ❖ WBS   | Work Breakdown Structure                          |
| ❖ XML   | Extensible Markup Language                        |
| ❖ XPath | XML Path Language                                 |

# Chapter One: Introduction

## Objectives

*At the end of this chapter students will be able to*

- ☞ Comprehend the concepts of Software Quality, quality assurance and Standards.
- ☞ Identify software quality factors and software measures
- ☞ Model a Software Quality Assurance Framework & standards for analyzing quality metrics.
- ☞ Develop product and process quality metrics using different principles.
- ☞ Construct test cases using variety test strategies for any given applications.
- ☞ Design an appropriate testing methodology & tool for any software requirements.

## 1.1 Software Quality

Software quality is conformance to explicitly stated functional and performance requirements, explicitly documented development standards and implicit characteristics that are expected of all professionally develop software. Three (3) important points to remember on software quality.

- ❖ Software requirements are the foundation from which quality is measured. Lack of conformance to requirements is lack of quality.
- ❖ Specified standards define a set of development criteria that guide the manner in which software is engineered. If the criteria are not followed, lack of quality will almost surely result.
- ❖ There is a set of implicit requirements that often goes unmentioned (e.g., the desire for good maintainability). If software conforms to its explicit requirements, but fails to meet implicit requirements.

### ■ IEEE Definition:

- ❖ It is the degree to which a system, component, or process meets specified requirements.
- ❖ The degree to which a system, component, or process meets customer or user needs or expectations.
- ❖ In order to determine whether a system, system component, or process is of high quality we use what are called quality attributes. the degree to which they possess a given quality attribute with quality metrics.

### ■ According to Philip B. Crosby (1979):

- ❖ “Quality means conformance to requirements”

Analysis:

- ❖ Crosby’s definition of software quality refers to the degree to which the written software meets the specifications prepared by the customer and his professional team. This means that errors included in the software specification are not considered and do not reduce the software quality, a characteristic that can be considered the approach’s deficiency.

### ■ According to Joseph M. Juran(1988):

- ❖ Quality consists of those product features which meet the needs of customers and thereby provide product satisfaction and quality consists of freedom from deficiencies”

Analysis:

- ❖ Juran's definition is aimed at achieving customer satisfaction and views the fulfillment of customers' real needs as the true goal of software quality.

## **1.2 Quality Characteristics**

Quality characteristics is any property or element that can be used to define the nature of a product. Each characteristic can be physical or chemical properties such as size, weight, volume, color or composition.

### **Software Quality**

- ❖ Is achieved through a disciplined approach - called software engineering SE
- ❖ Can be defined, described, and measured
- ❖ Can be assessed before any code has been written
- ❖ Cannot be tested into a product

### **Software Quality Challenges**

- ❖ Defining it
- ❖ Describing it (qualitatively)
- ❖ Measuring it (quantitatively)
- ❖ Achieving it (technically)
- ❖ Designing software is a creative task, and like most such tasks, success is more likely if the designer follows what might be termed a set of rules of form. The rules of form also provide some way of assessing the quality of the eventual product, and possibly of the processes that led to it.

## **1.3 Software Quality Activities**

SQA is ensured through a Quality Management System (QMS), QMS is made of several components; it is a system integrated in the bigger system of software development, which comprises project, process and product management systems.

The Software Engineering Institute (SEI) recommends a set of activities, which, when implemented effectively, assures the designed quality. These activities include:

- Quality assurance planning
- Data gathering on key quality defining parameters
- Data analysis and reporting

- Quality control mechanisms

The first and foremost requirement in SQA is that it is a separate group responsible for quality in the organization. They set the goals, standards and mechanisms (systems) for SQA. The role of the SQA group is to assist the software development team in managing the quality requirements of the software. Every software has certain quality goals specified by the customer. These quality goals are to be achieved by the development team by introducing a set of activities or ensuring the delivery of quality to the customer.

SQA activities operate on the normal activities of quality management. These activities play the role of monitoring, tracking, evaluations, auditing and reviews to ensure that the quality policy of the organization is implemented. These activities are independently carried out, and feedback is given to the development team. The responsibility of delivering the required quality to the customer rests with the development team. The development team has an obligation to implement quality policy in terms of goals, objectives, procedures, checks and controls, documentation and feedback to management. For example, the quality policy stipulates preparation of a test plan for stages for development as well as at the end of the development process. SQA has a variety of tools to implement the policy. They are

- ❖ Auditing

- ❖ Inspection

Verify compliance with those norms and practices specified in QA policy; deviations are set right. Ensure that deviations are documented and reported and put into the QA database for guidance. Design and architecture is reviewed to ensure that standards are met and customer quality is assured. Implement change management. Collect data on various observations in the process of auditing, inspection and reviews to build QA database and to improve various standards. **Software Defects**

The causes for not meeting the quality commonly are

- ❖ Imprecise requirement and software specifications
- ❖ Lack of understanding of customer requirements
- ❖ International deviations
- ❖ Violation of standards (Design, Programming)

- ❖ Erroneous data representation
- ❖ Improper interface
- ❖ Faulty logic in rules and processes
- ❖ Erroneous testing
- ❖ Incomplete and defective documentation
- ❖ H&S platforms not coping up to required standards
- ❖ Lack of domain knowledge

Statistical analysis of errors or defects helps to focus and concentrate on probable areas where SQA efforts are necessary. SQA is also concerned with two other aspects namely, software reliability and software safety. Software reliability is defined as the probability of failure free operation of a computer program in a specified environment for a specified time. The nature of failure may be such that one error may require only a few repairs, and other may need hours. SQA collects data on these failures and examines why these failures could not be prevented through earlier SQA activities.

A simple measure of reliability is Mean Time between Failure (MTBF).

$$MTBF = MTTF + MTTR$$

Where MTTF is Mean Time to Failure and MTTR is Mean Time to Repair.

Software safety deals with identification and assessment of potential hazards of software failing, and its impact on the system or in the environment in which it operates. Software safety needs are more crucial in process control systems, health care systems, defense and so on. SQA activities concentrate on such areas of software where failure affects the customer system adversely. In short, SQA efforts assure software quality, reliability, availability and safety.

#### **1.4 Software Quality Assurance**

Software quality assurance is planned and systematic pattern of all actions necessary to provide adequate confidence that an item or product conforms to established technical requirements. A set of activities designed to evaluate the process by which the products are developed or manufactured. Contrast with quality control.

## Objective of Software Quality Assurance

- ❖ Assuring with an acceptable level of confidence that the software maintenance activities will conform to the functional technique requirements.
- ❖ Assuring with an acceptable level of confidence that the software maintenance activities will conform to managerial scheduling and budgetary requirements

### 1.5 Software Quality Factor

The requirements document is one of the most important elements for achieving software quality. The great variety of issues related to the various attributes of software and its use and maintenance, as defined in software requirements documents, can be classified into content groups called quality factors.

#### **Classifications of software requirements into software quality factors:**

Several models of software quality factors and their categorization in factor categories have been suggested over the years. The classic model of software quality factors, suggested by McCall, consists of 11 factors [1]. Subsequent models, consisting of 12 to 15 factors, were suggested by Deutsch and Willis (1988) and by Evans and Marciniak (1987). The alternative models do not differ substantially from McCall's model. The McCall factor model, despite the quarter of a century of its "maturation", continues to provide a practical, up-to-date method for classifying software requirements (Pressman, 2000).

#### **McCall's factor model**

McCall's factor model classifies all software requirements into 11 software quality factors. The 11 factors are grouped into three categories – product operation, product revision and product transition – as follows:

- ❖ **Product operation factors:** Correctness, Reliability, Efficiency, Integrity, Usability.
- ❖ **Product revision factors:** Maintainability, Flexibility, Testability.
- ❖ **Product transition factors:** Portability, Reusability, Interoperability.

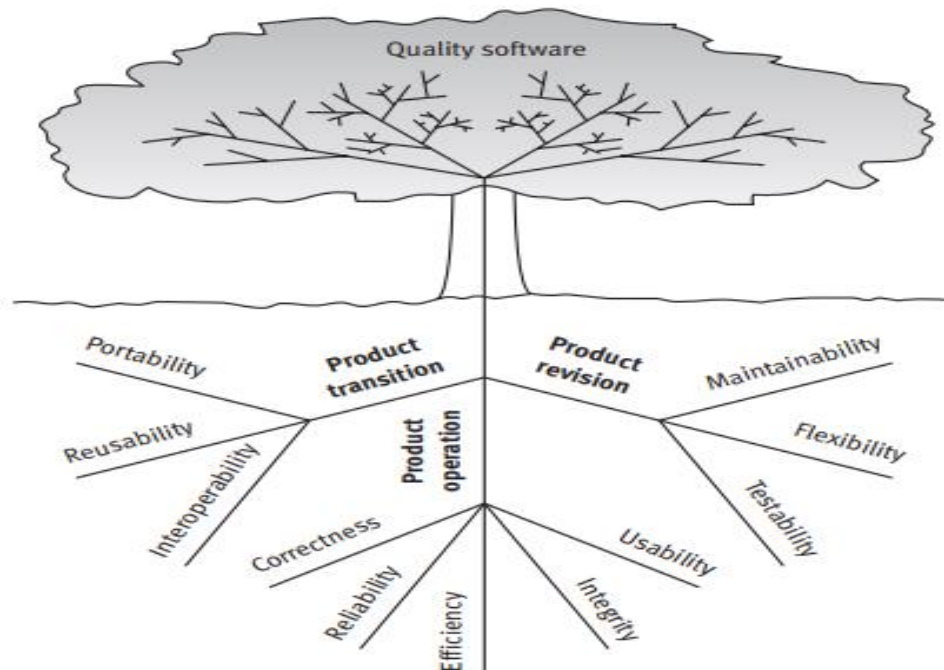


Figure 1:McCall's quality factor model tree

### 1.5.1 Product operation software quality factors

Product operation software quality factors deal with requirements that directly affect the daily operation of the software. According to McCall's model, five software quality factors are included in the product operation category, all of which deal with requirements that directly affect the daily operation of the software. These factors are as follows.

#### **Correctness (*Does it do what I want?*)**

The extent to which a program satisfies its specification and fulfills the customer's mission objectives. The extent to which software is free from design defects and from coding defects.; that is fault-free. Correctness requirements are defined in a list of the software system's required outputs, such as a query display of a customer's balance in the sales accounting information system, or the air supply as a function of temperature specified by the firmware of an industrial control unit.

#### *Example*

The correctness requirements of a club membership information system consisted of the following:

- ❖ The output mission: A defined list of 11 types of reports, four types of standard letters to members and eight types of queries, which were to be displayed on the monitor on request.
- ❖ The required accuracy of the outputs: The probability for a non-accurate output, containing one or more mistakes, will not exceed 1%.
- ❖ The completeness of the output information: The probability of missing data about a member, his attendance at club events, and his payments will not exceed 1%.
- ❖ The up-to-dateness of the information: Not more than two working days for information about participation in events and not more than one working day for information about entry of member payments and personal data.
- ❖ The availability of information: Reaction time for queries will be less than two seconds on average; the reaction time for reports will be less than four hours.
- ❖ The required standards and guidelines: The software and its documentation are required to comply with the client's guidelines.

### **Reliability (*Does it do it accurately all of the time?*)**

The extent to which a program can be expected to perform its intended function with required precisions under stated conditions for a stated period of time. This requirement deals with failures to provide service. They determine the maximum allowed software system failure rate, and can refer to the entire system or to one or more of its separate functions.

#### *Examples*

- ❖ The failure frequency of a heart-monitoring unit that will operate in a hospital's intensive care ward is required to be less than one in 20 years. Its heart attack detection function is required to have a failure rate of less than one per million cases.
- ❖ One requirement of the new software system to be installed in the main branch of Independence Bank, which operates 120 branches, is that it will not fail, on average, more than 10 minutes per month during the bank's office hours. In addition, the probability that the off-time (the time needed for repair and recovery of all the bank's services) be more than 30 minutes is required to be less than 0.5%.

### **Efficiency (*Will it run on my hardware as well as it can?*)**

The extent to which a software performs its function with a minimum consumption of computing resources. It should not use any hardware components or peripheral equipment

unnecessarily. Efficiency requirements deal with the hardware resources needed to perform all the functions of the software system in conformance to all other requirements. The main hardware resources to be considered are the computer's processing capabilities (measured in MIPS – million instructions per second, MHz or megahertz – million cycles per second, etc.), its data storage capability in terms of memory and disk capacity (measured in MBs – megabytes, GBs – gigabytes, TBs – terabytes, etc.) and the data communication capability of the communication lines (usually measured in KBPS – kilobits per second, MBPS – megabits per second, and GBPS – gigabits per second). The requirements may include the maximum values at which the hardware resources will be applied in the developed software system or the firmware.

### **Integrity (is it secure?)**

Integrity requirements deal with the software system security, that is, requirements to prevent access to unauthorized persons, to distinguish between the majority of personnel allowed to see the information (“read permit”) and a limited group who will be allowed to add and change data (“write permit”), and so forth.

### **Usability (is it designed for the use?)**

The effort required to learn, operate, prepare input, and interpret output of a program.

#### *Example*

The software usability requirements document for the new help desk system initiated by a home appliance service company lists the following specifications:

- ❖ A staff member should be able to handle at least 60 service calls a day.
- ❖ Training a new employee will take no more than two days (16 training hours), immediately at the end of which the trainee will be able to handle 45 service calls a day.

### **1.5.2 Product revision software quality factors**

According to the McCall model of software quality factors, three quality factors comprise the product revision category. These factors deal with those requirements that affect the complete range of software maintenance activities: corrective maintenance (correction of software faults and failures), adaptive maintenance (adapting the current software to additional circumstances and customers without changing the software) and perfective maintenance (enhancement and improvement of existing software with respect to locally limited issues). These are as follows.

**Maintainability (*can I fix it?*)**

Maintainability requirements determine the efforts that will be needed by users and maintenance personnel to identify the reasons for software failures, to correct the failures, and to verify the success of the corrections. This factor's requirements refer to the modular structure of software, the internal program documentation, and the programmer's manual, among other items.

**Flexibility (*can I change it?*)**

The capabilities and efforts required to support adaptive maintenance activities are covered by the flexibility requirements. These include the resources (i.e. in man-days) required to adapt a software package to a variety of customers of the same trade, of various extents of activities, of different ranges of products and so on. This factor's requirements also support perfective maintenance activities, such as changes and additions to the software in order to improve its service and to adapt it to changes in the firm's technical or commercial environment.

**Testability (*Can I test it*)**

Testability requirements deal with the testing of an information system as well as with its operation. Testability requirements for the ease of testing are related to special features in the programs that help the tester, for instance by providing predefined intermediate results and log files. Testability requirements related to software operation include automatic diagnostics performed by the software system prior to starting the system, to find out whether all components of the software system are in working order and to obtain a report about the detected faults. Another type of these requirements deals with automatic diagnostic checks applied by the maintenance technicians to detect the causes of software failures.

**1.5.3 Product transition software quality factors**

According to McCall, three quality factors are included in the product transition category, a category that pertains to the adaptation of software to other environments and its interaction with other software systems.

**Portability (*Will I be able to use it on another machine?*)**

Portability requirements tend to the adaptation of a software system to other environments consisting of different hardware, different operating systems, and so forth. These requirements

make it possible to continue using the same basic software in diverse situations or to use it simultaneously in diverse hardware and operating systems situations.

*Example*

A software package designed and programmed to operate in a Windows 2000 environment is required to allow low-cost transfer to Linux and Windows NT environments.

**Reusability (Will I be able to reuse some of the software?)**

Reusability requirements deal with the use of software modules originally designed for one project in a new software project currently being developed. They may also enable future projects to make use of a given module or a group of modules of the currently developed software. The reuse of software is expected to save development resources, shorten the development period, and provide higher quality modules. These benefits of higher quality assume that most of the software faults have already been detected by the quality assurance activities performed on the original software, by users of the original software, and during its earlier reuses. The issues of software reuse became a subject of software industry standards (see IEEE, 1999).

**Interrogability (Will I be able to interface it with another system?)**

Interoperability requirements focus on creating interfaces with other software systems or with other equipment firmware (for example, the firmware of the production machinery and testing equipment interfaces with the production control software). Interoperability requirements can specify the name(s) of the software or firmware for which interface is required. They can also specify the output structure accepted as standard in a specific industry or applications area.

**1.6 Software Quality Assurance Components**

The SQA Components that are used by the Software Quality Assurance System can be classified into

six different categories; each of which is necessary to guarantee maximum quality and to ensure the compliance with the standards and procedures.

“The environment in which software development and maintenance is undertaken directly influences the SQA components. Alongside this various error will also affect the SQA components; therefore, it is usually necessary to include all of the components.”

The six different components are broken down into the following categories

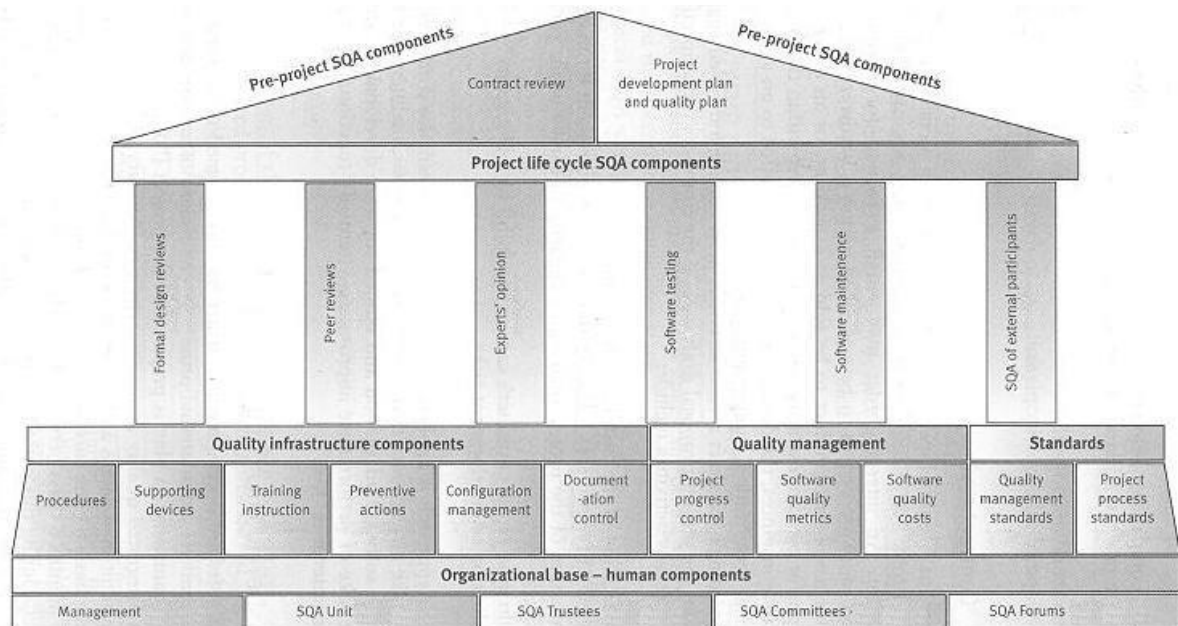


Figure 2:SQA System Overview

### 1.6.1 Pre-project Components

The Pre-Project component ensures that the project has been adequately defined ensuring that the resources available, budget and so forth have not been misinterpreted by the client or the organization. This improves the preliminary steps taken prior to initiating work on the project itself, thus leading to fewer errors later in the development phase. There are two key concepts to this component;

#### A. Contract Review

The Contract Review begins the negotiations of a contract between the company and the client. One of the key focuses is on the areas or points that could go wrong, known as development risks. The contract ensures that commitments have been documented including an agreement upon the functional specification, budget and the schedule.

The contract review activities must include a detailed examination of the project proposal draft

and the contract draft, each of which have different objectives, to ensure that the commitments have been properly and clearly defined. A checklist is often used by reviewers to make this stage easier and it is expected that the review stage will occur more than once.

The outcome from this stage is a documented agreement between the company and client stating the projects requirements, budgets and so on ensuring that no new changes have been added to the contract draft.

## **B. Development and Quality Plans**

The development and Quality Plans stage occurs once an agreement has been made and a software development contract has been signed. The time between a proposal, the contract review and the signing of the contract could be prolonged and during this time changes may occur. Consequently, proposal materials need to be revised and new plans are required; a development plan and a quality plan.

The development plan focuses on eleven specific elements, some of which are; the products of the project i.e. software products and design documents specifying deliverables. Project interfaces detailing what, if any, interfaces the product will have with existing software packages. The methodology clearly states which methodology should be used at each phase of the project.

The quality plan has four main elements; however, how many of these are used is dependent upon the project. Quality Goals refers to the goals of the product, it is always better to have more goals as it is easier to see how well the system performs. Planned review activities are a listing of all the planned reviews such as Design Reviews (DR) and Code Inspections. Planned software tests details all the software tests that are to be performed with details such as the unit, integration or complete system to be tested. Finally, planned acceptance tests for externally developed software; this is only necessary for products not developed in-house.

Approval of the two plans is necessary and is approved or rejected according to the procedures within the organization.

### 1.6.1 Project Lifecycle Activities and Assessment

This component has two main stages; 1. The Development Lifecycle stage which aims to detect design and programming errors. 2. The Operation-Maintenance stage focuses on maintenance tasks that improve functionality.

When developing the product there are a number of activities that take place, these are reviews, such as formal design reviews and peer reviews, expert opinions, software testing, software maintenance and finally, the assurance of the quality of external participants' work which ensures that any efforts by external members meet the quality and standards of the organization.

The reviews occur during the design phase of the development process and should be conducted at any milestone. The formal Design Review (DR) is a very important document as the project cannot continue until a formal approval by the DR committee has been received. The document itself includes a list of action items (corrections) that are required. The peer review, guided by checklists, standards and past problems, is a document that reviews short documents or sections to attempt to detect design and programming faults documenting a list of its findings.

'Expert Opinions' is the use of an external member to review in-house work; it can reinforce the internal (in-house) quality assurance activities. Although not all organizations may use this approach it is useful for small organizations which may not have sufficient professional capabilities in-house or for a replacement to the regular in-house professional for whatever reasons. The outcome from this is often a document similar to a formal design review.

*"Software Testing is a formal process carried out by a specialized team in which a software unit, several integrated software units or an entire software package are examined by running the programs on a computer. All associated tests are performed according to the approved test procedures on approved test cases"* D.Galin (2003)

In general, Software Testing is aimed at reviewing the running and functionality of the software in which the testing is based on a prepared list of test cases.

Software Maintenance specifies three categories in which maintenance for a system can fall into. The first is corrective maintenance in which the correction of failed software, for whatever reason, occurs. The second is adaptive maintenance in which maintenance occurs to the system it needs to be adapted; however, the basic functionality of the system is left

untouched. Finally, the third is functionality improvement where the system is modified to add improvements, this could significantly change the system compared to the other two points.

### **1.6.2 Infrastructure components for error prevention and Improvement**

The main goal of this component is to reduce the number of errors in the system and improve productivity. This is achieved through the use of a number of sub-components.

#### **A. Procedures and Work Instructions**

A procedure describes how the process, a specific development activity, is performed whilst a work instruction is more specific and provides detailed directions for the use of methods. Having a detailed guideline ensures that all members know how to achieve some goal.

#### **B. Supporting Quality Devices**

‘Quality Devices’ are used to maximize efficiency and quality. A quality device could be a template or a checklist which saves time as the document will be complete and will not have to be developed, from scratch each time.

Using Quality Devices offers an improved form of communication and provides standardization within the organization as each document will be of the same nature.

#### **C. Staff Training, Instruction and Certification**

Staff training is a vital element to avoiding errors throughout the development of the system. Having well trained professional staff enables efficient and high-quality performance from each member. New staff have to be trained in respect to the standards and procedures of the organization and existing staff should be re-trained when assignments change.

#### **D. Preventive and Corrective Actions**

Studying existing data for similar faults and failures can enable future ones to be solved easily either by correcting it once it has occurred or by preventing it from happening.

The data should be recorded, or found, in design review reports, software test reports or customer complaints. It should be managed effectively so that if it occurs in the future the solution, if one was found, can be easily accessible.

### **E. Configuration Management**

This deals with the dangers of version releases. With intense work focusing on new versions and new software releases dangers arise when different members carry out the same tasks. This can lead to misidentification of the versions or releases or loss of records or development activities.

Configuration management introduces procedures that are used to control the change process and monitor it.

### **F. Documentation Control**

Document Control is necessary to ensure long term availability of controlled documents. Controlled documents are maintained and updated. They are formally approved and contain evidence of the systems performance.

## **1.7 Software Quality Management**

Quality management not only focuses on product quality but also the means in which it can be achieved. Some of the components used to support the managerial control of software development projects are the Project Progress Control, Software Quality Metrics and Software Quality costs.

The aim of Project Progress Control is simply to monitor the progress of the project to ensure that it does not deviate from its initial plans. It focuses particularly on monitoring resource usages, schedules (whether they are being met), risk management activities and the budget.

A Software Quality Metric is a measurement that is used to evaluate software quality in a system. The software is the input, and the output is a numeric value which represents the **degree to which the software possesses a given quality attribute**. The measures can apply to functional quality, productivity and the organization side of the project. The Software Quality Costs are the costs that incur throughout the entire project; the total cost is calculated from the costs of control and the costs of failure. The organizations main interest is the sum of the total costs which will determine a success or failure.

## Review Questions

1. Which of the following is not categorized under Product Operation of McCall's Software Quality Factors?
 

A. Efficiency

B. Reliability

C. Integrity

D. Reusabilit
2. The software requirement document regarding a software system for managing a hospital laboratory (Super Lab) consists of chapters of requirements. For each section, fill in the name of the factor that best fits the requirement (use McCall's factor model and the attached reading list to choose only one factor per requirements section).

| No | Section taken from the software requirement document                                                                                                                                                                                                                                                                                                                                                                                              | Quality factors |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| 1  | The probability that the "Super-lab" software system will be found in a state of failure during peak hours (9 am to 4 pm) is required to be below 0.5%.                                                                                                                                                                                                                                                                                           |                 |
| 2  | The "Super-Lab" software system will enable direct transfer of laboratory results to those files of hospitalized patients managed by the "MD-File" software package.                                                                                                                                                                                                                                                                              |                 |
| 3  | The "Super-Lab" software system will include a module that prepares a detailed report of the patient's laboratory test results during his current hospitalization. (This report will serve as an Appendix to the family physician's file). The time required to obtain this printed report will be less than 30 seconds; the level of accuracy and completeness will be at least 99%.                                                             |                 |
| 4  | The "Super-Lab" software to be developed for hospital laboratory use may be adapted later for private laboratory use.                                                                                                                                                                                                                                                                                                                             |                 |
| 5  | The training of a laboratory technician, requiring no more than 3 days, will enable the technician to reach level C of "Super-Lab" software usage. This means he or she will be able to manage reception of 20patients per Hour.                                                                                                                                                                                                                  |                 |
| 6  | The "Super-Lab" software system will record a detailed user 's Log. In addition, the system will report attempts by unauthorized persons to obtain medical information from the laboratory test results data base. The report will include the following information's: The network identification of the applying terminal, the system code of the employee who requested that information, the day and time of attempt and the type of attempt. |                 |
| 7  | The "Super-Lab" sub-system that deals with billing patients for their tests may be eventually used as a subsystem in the "physiotherapy center" software package.                                                                                                                                                                                                                                                                                 |                 |

|    |                                                                                                                                                                                                                 |  |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| 8  | The different system components should be kept so simple as possible, and very well documented.                                                                                                                 |  |
| 9  | The software system should be able to serve 12 workstations and 8 automatic testing machines with a single model AS20 server and a cs25 communication server that will be able to serve 25 communication lines. |  |
| 10 | The “Super-Lab” software package developed for the Linux Operating System should be compatible for applications in a window NT environment.                                                                     |  |
| 11 | The system software documentation should be clear, self-descriptive, and have a high degree of consistency.                                                                                                     |  |

3. From McCall's software Quality factor, which one will best fit the following scenario? Justify your answer.
  - A. The test planner describes recruiting a new quality assurance team will need and he suggests that training a new employee will take no more than 7 days training hours), immediately at the end of which the trainee will be able to handle test log results.
  - B. An unanticipated external message is received by a process during normal operation. The process informs the operator of the receipt of the message and continues to operate with no downtime.
4. List out the roles of software Quality Assurance engineer?
5. What is the difference between Quality Control and Quality Assurance?
6. What is the role of quality assurance in a product development lifecycle?

## Chapter Two: Software Testing

### Objectives

*At the end of this chapter students will be able to*

- ☞ *Comprehend the concepts of Software Quality, quality assurance and standards.*
- ☞ *Study fundamental concepts in software testing*
- ☞ *Identify the Objective of software Testing*
- ☞ *Identify the type of software testing*
- ☞ *Apply effective testing techniques to test a software system*
- ☞ *Design specific and measurable test cases to ensure software requirements.*
- ☞ *Demonstrate the integration testing which aims to uncover interaction and compatibility problems as early as possible*
- ☞ *Integrate testing processes within a continuous delivery model of software development*

## 2.1 Software Testing Terminology

### ■ Test Case

A test case, in a practical sense, is a test-related item which contains the following information:

1. **A set of test inputs:** These are data items received from an external source by the code under test. The external source can be hardware, software, or human.
2. **Execution conditions:** These are conditions required for running the test, for example, a certain state of a database, or a configuration of a hardware device.
3. **Expected outputs:** These are the specified results to be produced by the code under test.

Example:

- ✓ Name: Search real estate test case
- ✓ Purpose: To test search functionality for a given real estate.

Table 1: Search real estate test case

| Input                            | Expected Results                        | Actual Results | Pass/Fail |
|----------------------------------|-----------------------------------------|----------------|-----------|
| Empty search field               | The system should indicate error        | As expected    | Pass      |
| Wrong value is inserted          | The system should indicate error        | As expected    | Pass      |
| Correct search filed is inserted | The system should display what is found | As expected    | Pass      |

### ■ Test Set:

A group of related tests is sometimes referred to as a test set.

### ■ Test Suite:

Test Suite is a high-level concept, grouping together hundreds or thousands of tests related by what they are intended to test

### ■ Test Log:

Defines the result of test cases in terms of passed/failed after execution of the test case on the application build.

### ■ Test Bed:

A test bed is an environment that contains all the hardware and software needed to test a software component or a software system. This includes the entire testing environment, for example, simulators, emulators, memory checkers, hardware probes, software tools, and all other items needed to support execution of the tests.

#### ■ **Test Oracle:**

A test oracle is a document, or piece of software that allows testers to determine whether a test has been passed or failed. Examples include a specification (especially one that contains pre- and post-conditions), a design document, and a set of requirements.

A program, or a document that produces or specifies the expected outcome of a test, can serve as an oracle. Examples include a specification (especially one that contains pre- and post-conditions), a design document, and a set of requirements. Other sources are regression test suites. The suites usually contain components with correct results for previous versions of the software. If some of the functionality in the new version overlaps the old version, the appropriate oracle information can be extracted. A working trusted program can serve as its own oracle in a situation where it is being ported to a new environment. In this case its intended behavior should not change in the new environment.

#### ■ **Error:**

An error is a mistake, misconception, or misunderstanding on the part of a software developer. In the category of developer, we include software engineers, programmers, analysts, and testers. For example, a developer may misunderstand a design notation, or a programmer might type a variable name incorrectly.

#### ■ **Faults**

A fault (defect) is introduced into the software as the result of an error. It is an anomaly in the software that may cause it to behave incorrectly, and not according to its specification. **Faults or defects are sometimes called —bugs.** Use of the latter term trivializes the impact faults have on software quality. Use of the term —defect is also associated with software artifacts such as requirements and design documents. Defects occurring in these artifacts are also caused by errors and are usually detected in the review process.

#### ■ **Failures**

A failure is the inability of a software system or component to perform its required functions within specified performance requirements. During execution of a software component or system, a tester, developer, or user observes that it does not produce the expected results.

## 2.2 Observation about Testing

- ❖ “Testing is the process of executing a program with the intention of finding errors ”. **Myers**
- ❖ “Testing can show the presence of bugs but never their absence” **Dijkstra**
- ❖ According to **ANSI/IEEE 1059** standard, Testing can be defined as A process of analyzing a software item to detect the **differences between existing and required conditions** (that is defects/errors/bugs) and to **evaluate the features of the software** item.
- ❖ Testing ensures that the software behaves and looks exactly like what is mentioned in the requirements specification document, so that when software is delivered to the client there are no arguments about the variation from the original requirements.
- ❖ Testing is the process to execute the program and find the bugs and fix on the spot. It evaluates the quality of the software, and done during the development and after the implementation process.

## 2.3 Objective of Testing

The main aim of testing is to analyze and examine the performance of a software and to evaluate the errors that occurred during the execution of a software, when the software is running with different inputs in different environment.

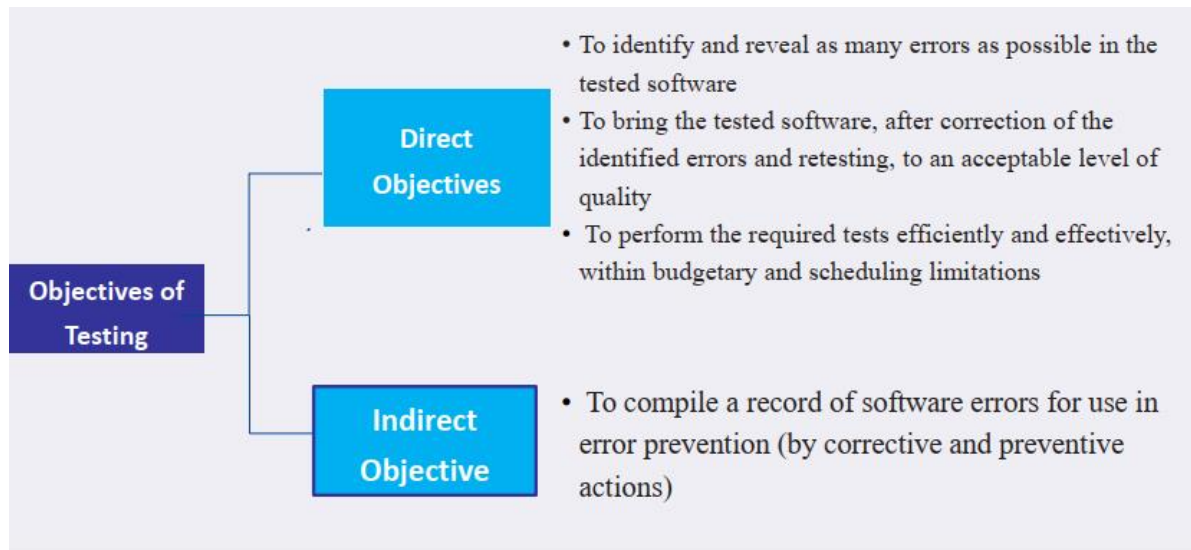


Figure 3: Objective of software Testing

### When to start testing

An early start to testing reduces the cost, time to rework and error free software that is delivered to the client.

- However in Software Development Life Cycle (SDLC) testing can be started from the **Requirements** Gathering phase till the **deployment** of the software.
- However it also **depends on the development model that is being used.**
- For example, in Water fall model ,formal testing is conducted in **the Testing phase,**
- But in incremental model, testing is performed **at the end of every increment/iteration** and at the end the whole application is tested.

### When to stop testing

Unlike when to start testing, it is difficult to determine when to stop testing, as testing is a **never-ending process** and no one can say that any software is 100% tested. Following are the aspects which should be considered to stop the testing:

- **Completion of Functional and code coverage to a certain point.**
- 🐛 **Bug rate falls below a certain level and no high priority bugs are identified.**

## 2.4 Fundamental principles of testing

- ❖ The goal of testing is to find defects before customers find them out.
- ❖ Exhaustive testing is not possible; program testing can only show the presence of defects, never their absence.
- ❖ Testing applies all through the software life cycle and is not an end of-cycle activity.
- ❖ Understand the reason behind the test.
- ❖ Tests develop immunity and have to be revised constantly.
- ❖ Defects occur in convoys or clusters, and testing should focus on these convoys.
- ❖ Testing encompasses defect prevention.
- ❖ Testing is a fine balance of defect prevention and defect detection.
- ❖ Intelligent and well-planned automation is key to realizing the benefits of testing.
- ❖ Testing requires talented, committed people who believe in themselves and work in teams.

## 2.5 Software Testing Life Cycle – Different Stages of Testing

Software Testing Life Cycle (STLC) defines a series of activities conducted to perform Software Testing. It identifies what test activities to carry out and when to accomplish those test activities. In the STLC process, each activity is carried out in a planned and systematic way and each phase has different goals and deliverables.

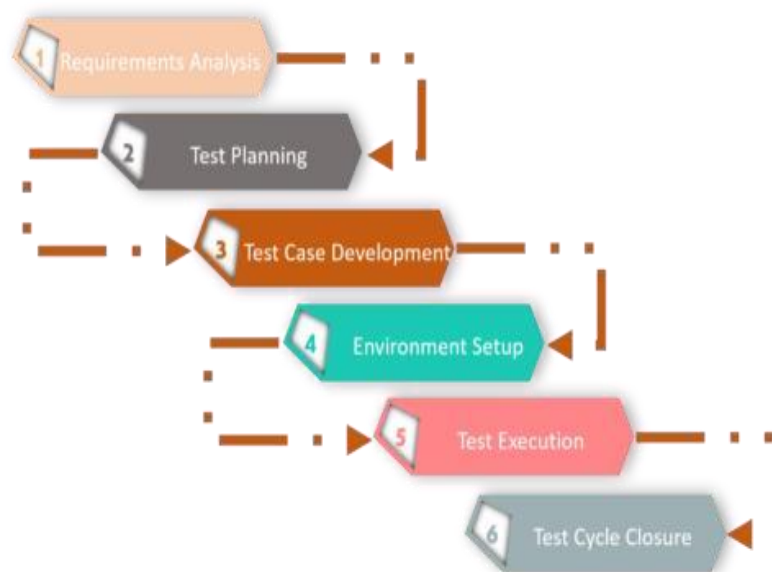


Figure 4: Software Testing Life Cycle

## **What are the different phases of Software Testing Life Cycle?**

The different phases of Software testing life cycle are:

- ❖ Requirement Analysis
- ❖ Test Planning
- ❖ Test Case Development
- ❖ Environment Setup
- ❖ Test Execution
- ❖ Test Cycle Closure

Now let's move ahead and have a look at the different phases of software testing life cycle in detail.

### **1. Requirement Analysis**

Requirement Analysis is the first step involved in Software testing life cycle. In this step, Quality Assurance (QA) team understands the requirement in terms of what we will testing & figure out the testable requirements. During this phase, test team studies the requirements from a testing point of view to identify the testable requirements. The QA team may interact with various stakeholders such as client, business analyst, technical leads, system architects etc. to understand the requirements in detail.

The different types of Requirements include:

**Business Requirements** – They are high-level requirements that are taken from the business case from the projects.

**Architectural & Design Requirements** – These requirements are more detailed than business requirements. It determines the overall design required to implement the business requirement.

**System & Integration Requirements** – It is detailed description of each and every requirement. It can be in form of user stories which is really describing everyday business language. The requirements are in abundant details so that developers can begin coding.

| Entry Criteria                                                                                                                                                    | Deliverable                                                                                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>The following documents are required:</p> <ul style="list-style-type: none"> <li>• Requirements Specification.</li> <li>• Application architectural</li> </ul> | <ul style="list-style-type: none"> <li>• List of questions with all answers to be resolved from testable requirements</li> <li>• Automation feasibility report</li> </ul> |

| Activities                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• Prepare the list of questions or queries and get resolved from Business Analyst, System Architecture, Client, Technical Manager/Lead etc.</li> <li>• Make out the list for what all Types of Tests performed like Functional, Security, and Performance etc.</li> <li>• Define the testing focus and priorities.</li> <li>• List down the Test environment details where testing activities will be carried out.</li> </ul> |

## 1. Test Planning

Test Planning is the most important phase of Software testing life cycle where all testing strategy is defined. This phase is also called as **Test Strategy** phase. In this phase, the Test Manager is involved to determine the effort and cost estimates for the entire project. It defines the objective & scope of the project. Test plan is one of the most important steps in software testing life cycle. The steps involved in writing a test plan include:

1. Analyze the product
2. Design Test Strategy
3. Define Test Objectives
4. Define Test Criteria

5. Resource Planning
6. Plan Test Environment
7. Schedule & Estimation
8. Determine Test Deliverable

### **Test Effort Estimation**

Lists out all the activities that have to be performed by the QA team and estimates how many man-hours each activity is going to take. The IEEE standard for test plan documentation does not include a separate cost component in its specification of a test plan. This is the usual case for many test plans since very often test costs are allocated in the overall project management plan. The project manager in consultation with developers and testers estimates the costs of testing. If the test plan is an independent document prepared by the testing group and has a cost component, the test planner will need tools and techniques to help estimate test costs. Test costs that should include in the plan are:

### **Process for test Estimation**

The following is the 4 Step process to arrive at an estimate

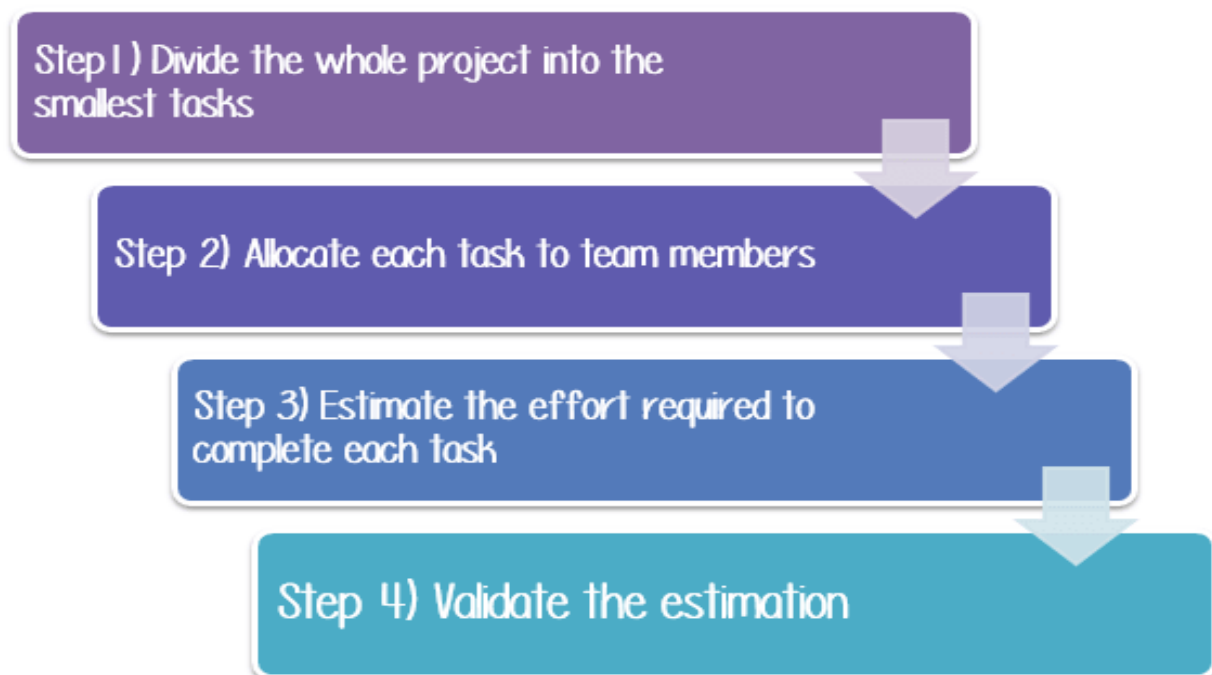
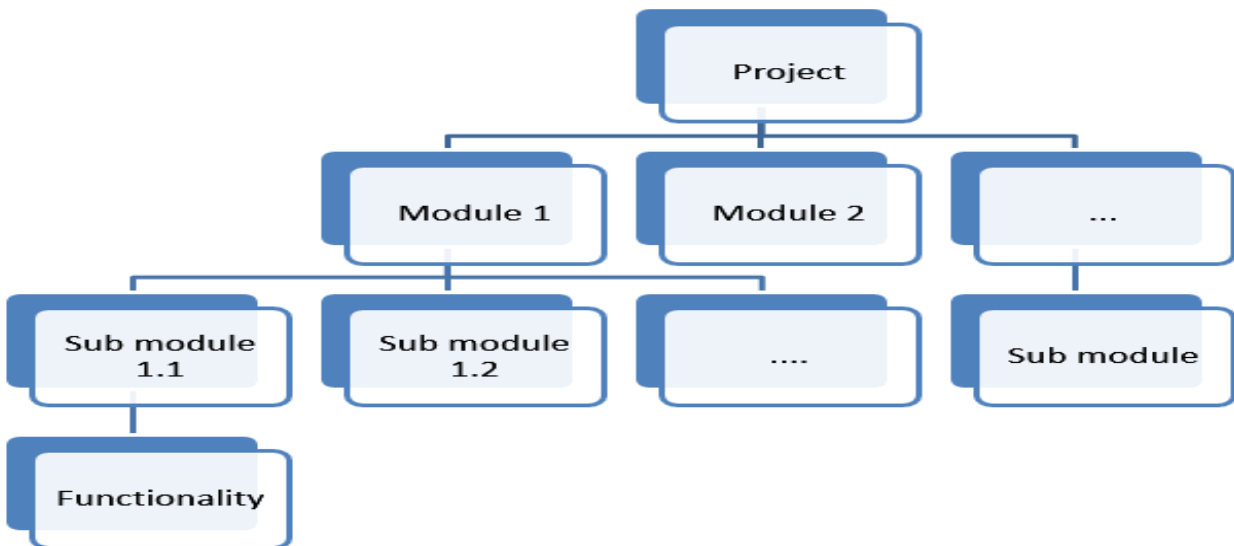


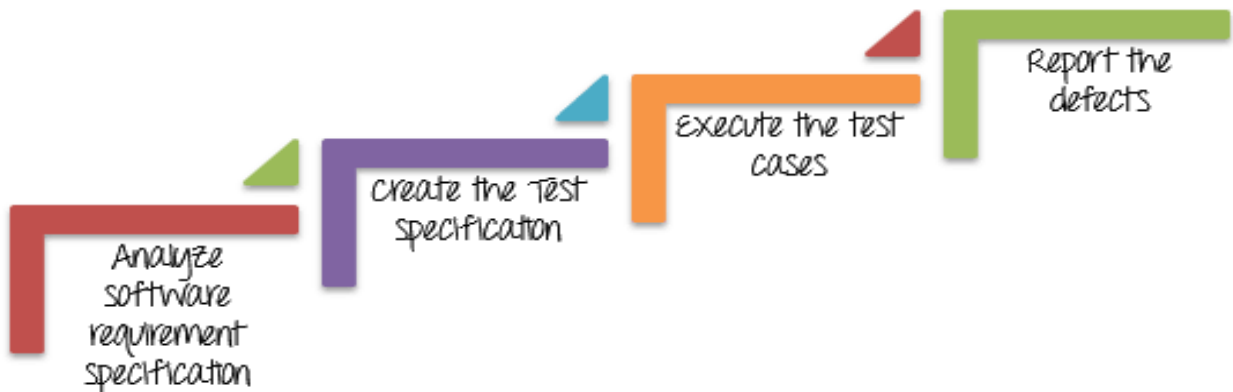
Figure 5: Test Estimation Process

### Step 1: Divide the whole project into smallest task

Task is a piece of work that has been given to someone. To do this, you can use the Work Breakdown Structure technique. In this technique, a complex project is divided into modules. The modules are divided into sub-modules. Each sub-module is further divided into functionality. It means divide the whole project task into the smallest tasks.



Example:



After that, we can break out each task to the subtask. The purpose of this activity is creating task as detailed as possible.

| Task                                       | Sub Tasks                                          |
|--------------------------------------------|----------------------------------------------------|
| Analyze software requirement specification | Investigate the software requirement specification |
|                                            | Interview with the developer & other stakeholders  |
|                                            | Design test scenarios                              |
| Create the Test Specification              | Create test cases                                  |
|                                            | Review and revise test cases                       |
| Execute the test cases                     | Build up the test environment                      |
|                                            | Execute the test cases                             |
|                                            | Review test execution results                      |
| Report the defects                         | Create the Defect reports                          |
|                                            | Report the defects                                 |

Table 1: Task to Sub Tasks

### Step 2: Allocate each task to a team member

| Task                                       | Members              |
|--------------------------------------------|----------------------|
| Analyze software requirement specification | All the Members      |
| Create the Test Specification              | Tester/ Test Analyst |
| Execute the test cases                     | Test Administrator   |
| Report the defects                         | Tester               |

Table 2: Allocate each task to a team member

### Step 3: Effort Estimation for Tasks

There are different techniques that you can apply to estimate the effort for tasks.

#### List of Software Test Estimation Techniques

- ❖ Work Breakdown Structure
- ❖ Functional point /Testing point analysis
- ❖ 3-point Software Test Estimation
- ❖ Delphi Technique

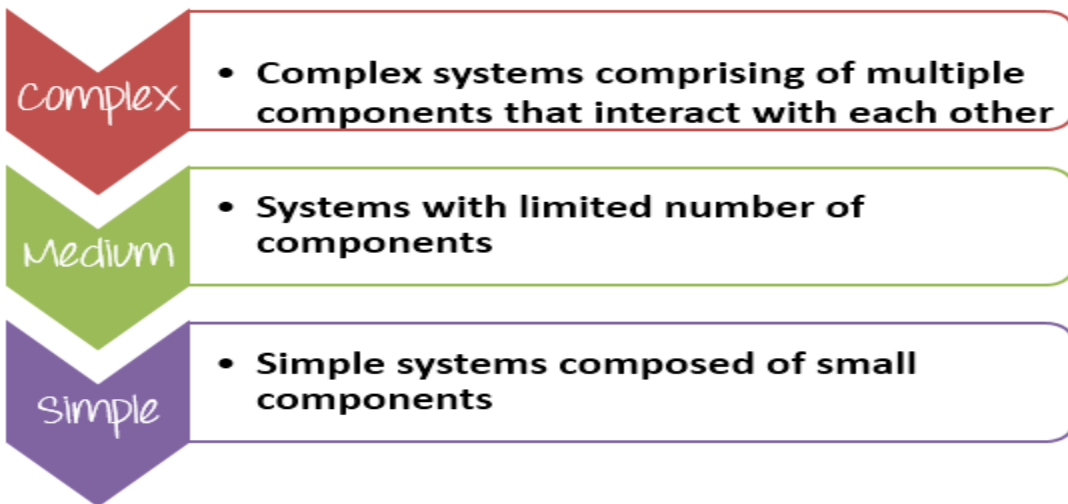
- ❖ Use-Case Method
- ❖ Percentage Distribution
- ❖ Ad-hoc Method

### Functional Point Method

using the Functional Point Method, the Manager estimates the Size, Duration, and cost of the task

#### ■ Step A: Estimate the Size of the Task

The size of this task depends on the functional size of the system under test. The functional size reflects the amount of functionality that is relevant to the user. The more the number of functionalities, the more complex system is. Prior to start actual estimating tasks effort, functional points are divided into three groups like **Complex**, **Medium** and **Simple** as following:



Based on the complex of software functions, the Test Manger has to give enough weightage to each functional point

| <u>No</u> | Module Name   | Role     | Description                                                                      | Weightage |
|-----------|---------------|----------|----------------------------------------------------------------------------------|-----------|
| 1         | Fund Transfer | Customer | Destination account.                                                             | 5         |
|           |               | Manager  | A manager can transfer funds from any source bank account to destination account |           |

|   |                 |          |                                                                                                     |   |
|---|-----------------|----------|-----------------------------------------------------------------------------------------------------|---|
| 3 | Deposit         | Manager  | A manager can deposit money into any account. Usually done when cash is deposited at a bank branch. | 3 |
| 4 | Change Password | Manager  | A manager can change password of only his account. He can't change passwords of his customers       | 1 |
|   |                 | Customer | A customer can change password of only his account                                                  |   |

Table 3: Assigning weightage to each Using functional point

### Step B: Estimate Duration for the Task

- ❖ After classifying the complexity of the function points, we have to estimate the duration to test them. Duration means how much time needs to finish the task.

$$\text{Total Effort} = \text{Total Function Points} * \text{Estimate defined per Function Points}$$

- ❖ **Total Effort:** The effort completely tests all the functionality
- ❖ **Total Function Points:** Total Modules of the project
- ❖ **Estimated defined per Function Points:** The average effort to complete one function points. This value depends on the productivity of the member who will take in charge this task.

|         | # of Functional Points | Weightage | Total |
|---------|------------------------|-----------|-------|
| Complex | 3                      | 5         | 15    |
| Medium  | 5                      | 3         | 15    |
| Simple  | 4                      | 1         | 4     |

|                                         |                     |
|-----------------------------------------|---------------------|
| Function Total Points                   | 34                  |
| Estimated Define per point              | 5                   |
| Total Estimated Effort (personal hours) | $34 \times 5 = 170$ |

Table 4: Estimate Duration for the Task

So, the total effort to complete the task of the example is around 170 man-hours

### Step C: Estimate the cost for the Tasks

- ❖ Suppose, on average the team salary is \$5 per hour. The time required for “Create Test Specs” task is 170 hours. Accordingly, the cost for the task is  $5 \times 170 = \$850$ .
- ❖ Now we can calculate budget for other activities in WBS and arrive at overall budget for the project
- ❖ The more accurate your estimate of project cost is, the better able you will be to manage your project’s budget.

### Step 4: Validate the Estimation

Once an aggregate estimate created for all the tasks mentioned in the WBS, we need to forward it to the management board, who will review and approve it. The management board will review and discuss the estimation plan with the one who done the estimation and explain to them the estimation logically and reasonably so that they can approve the estimation plan.

## 2. Test Case Development

The Test case development begins once the test planning phase is completed. This is the phase of STLC where testing team notes the detailed test cases. Along with test cases, testing team also prepares the test data for testing. Once the test cases are ready then these test cases are reviewed by peer members or QA lead.

A good test case is the one which is effective at finding defects and also covers most of the scenarios on the system under test. Here is the step by step guide on how to develop a good test case:

- ❖ Test cases need to be simple and transparent
- ❖ Create test case with end user in mind
- ❖ Avoid test case repetition

- ❖ Do not assume functionality and features of your software application
- ❖ Ensure 100% coverage of software requirements
- ❖ Name the test case is such that they are identified easily while tracking defects
- ❖ Implement testing techniques
- ❖ The test case you create must return the Test Environment to the pre-test state
- ❖ The test case should generate the same results every time
- ❖ Your peers should be able to uncover defects in your test case design

### 3. Test Environment Setup

Setting up the test environment is vital part of the Software Testing Life Cycle. A testing environment is a setup of software and hardware for the testing teams to execute test cases. It supports test execution with hardware, software and network configured.

Figure 3.1: Test Environment Setup



The test environment involves setting up of distinct areas like:

- ❖ **Setup of Test Server** – Every test may not be executed on a local machine. It may need establishing a test server, which can support applications.
- ❖ **Network** – We need to set up the network as per requirements.

- ❖ **Test PC Setup** – We need to set up different browsers for different testers.
- ❖ **Bug Reporting** – Bug reporting tools should be provided to testers.
- ❖ **Creating Test Data for the Test Environment** – Many companies use a separate test environment to test the software product. The common approach used is to copy production data to test.

| Entry Criteria                                                                                                                        | Deliverable                                                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• Test Plan or Test strategy document.</li> <li>• Test cases.</li> <li>• Test data.</li> </ul> | <ul style="list-style-type: none"> <li>• Test case execution report.</li> <li>• Defect report.</li> </ul> |

| Activities                                                                                                                                                                                                                                                                            |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• Mark status of test cases like Passed, Failed, Blocked, Not Run etc.</li> <li>• Assign Bug Id for all failed and blocked test cases.</li> <li>• Do Retesting once the defects are fixed.</li> <li>• Track the defects to closure.</li> </ul> |

#### 4. Test Execution

The next phase in Software Testing Life Cycle is Test Execution. Test execution is the process of executing the code and comparing the expected and actual results. When test execution begins, the test analysts start executing the test scripts based on test strategy allowed in the project.

#### 5. Test Cycle Closure

The final phase of the Software Testing Life Cycle is Test Cycle Closure. It involves calling out the testing team member meeting & evaluating cycle completion criteria based on Test coverage, Quality, Cost, Time, Critical Business Objectives, and Software. A test closure report by the test lead is published after accomplishing the exit criteria and finishing the testing phase. It follows a standard format such as:

- ❖ Test Summary Report
- ❖ Identifier
- ❖ Test Summary
- ❖ Variances
- ❖ Comprehensiveness Assessment
- ❖ Summary of Results
- ❖ Evaluation
- ❖ Summary of Activities
- ❖ Approval

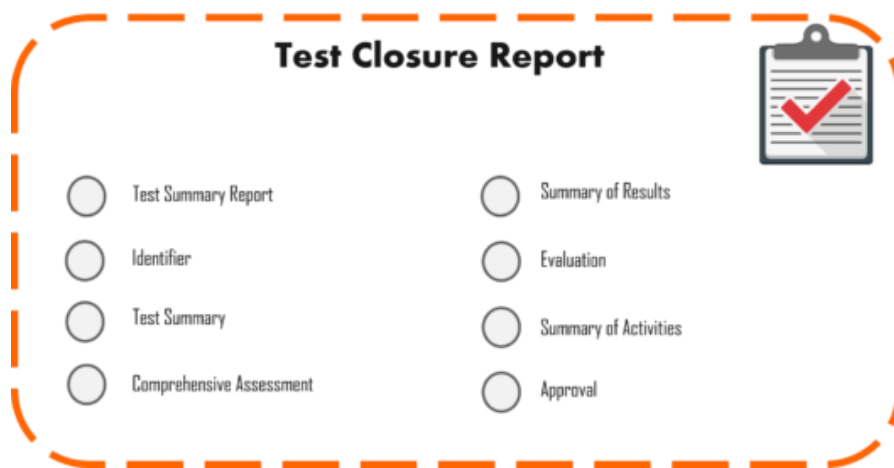


Figure 6: Stages of Test Closure

The process of test closure is implemented with the assistance of six important stages such as:

1. **Check planned Deliverable** – The planned deliverables that will be given to the stakeholder of the project are checked and analyzed by the team.
2. **Close Incident Reports** – The team checks that the planned deliverable are delivered and validates that all the incidents are resolved before the culmination of the process.
3. **Handover to Maintenance** – After resolving incidents and closing the incident report, the test-wares are then handed over to the maintenance team.
4. **Finalize & Archive Testware/Environment** – It involves finalizing and archiving of the testware and software like test scripts, test environment, test infrastructure, etc.
5. **Document System Acceptance** – It involves system verification and validation according to the strategy outlined.

6. **Analyze Best Practices** – It determines the various changes required for similar projects and their future release.

Let's now move ahead with this article and understand the difference between SDLC and STLC.

### *What is SDLC and STLC in Software Testing?*

| SDLC                                                                                                     | STLC                                                                                                 | 2.6 |
|----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|-----|
| Stands for Software Development Life Cycle                                                               | Stands for Software testing Life Cycle                                                               |     |
| It refers to a sequence of various activities that are performed during the software development process | It refers to a sequence of various activities that are performed during the software testing process |     |
| Aims to complete the development of the software including testing and other phases successfully         | Aims to evaluate the functionality of a software application to find any software bugs               |     |
| In SDLC, the code for the software is built based on the design documents                                | In STLC, the test environment is created and various tests are carried out on the software           |     |

### **Levels of Software of Testing**

Execution-based software testing, especially for large systems, is usually carried out at different levels. In most cases there will be **3-4 levels**, or major phases of testing: unit test, integration test, system test, and some type of acceptance test as shown in **Figure 6**.

Each of these may consist of one or more sublevels or phases. At each level there are specific testing goals. For example, at unit test a single component is tested.

- ❖ A principal goal is to detect functional and structural defects in the unit.
- ❖ At the integration level several components are tested as a group, and the tester investigates component interactions.
- ❖ At the system level the system as a whole is tested and a principle goal is to evaluate attributes such as usability, reliability, and performance.
- ❖ System test begins when all of the components have been integrated successfully. It usually requires the bulk of testing resources. Laboratory equipment, special software, or special hardware may be necessary, especially for real-time, embedded, or distributed systems. At the system level the tester looks for defects, but the focus is on evaluating performance, usability, reliability, and other quality-related requirements.

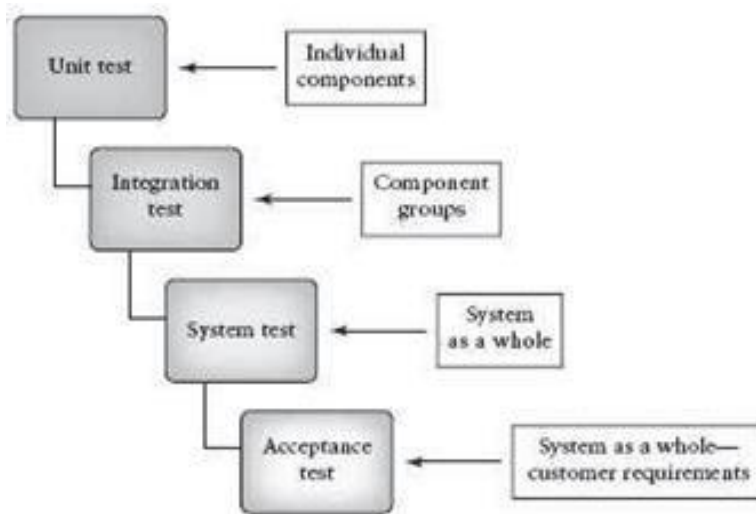


Figure 7: Level of Software Testing

The approach used to design and develop a software system has an impact on how testers plan and design suitable tests. There are two major approaches to system development bottom-up, and top- down.

These approaches are supported by two major types of programming languages:

- ❖ **Procedure- oriented**
- ❖ **Object-oriented.**

### 2.6.1 Unit Testing

A unit is the smallest possible testable software component. It can be characterized in several ways.

For example, a unit in a typical procedure-oriented software system:

- ❖ Performs a single cohesive function;
- ❖ can be compiled separately;
- ❖ Is a task in a work breakdown structure (from the manager's point of view);
- ❖ Contains code that can fit on a single page or screen.

A unit is traditionally viewed as a function or procedure implemented in a procedural (imperative) programming language. In object-oriented systems both the method and the class/object have been suggested by researchers as the choice for a unit. A unit may also be a small-sized **COTS**

(commercial off the shelf) component purchased from an outside vendor that is undergoing evaluation by the purchaser, or a simple module retrieved from an in-house reuse library.

## **1. Unit Test Planning**

A general unit test plan should be prepared. It may be prepared as a component of the master test plan or as a stand-alone plan. It should be developed in conjunction with the master test plan and the project plan for each project. Documents that provide inputs for the unit test plan are the project plan, as well the requirements, specification, and design documents that describe the target units. Components of a unit test plan are described in detail the **IEEE Standard for Software Unit Testing**. This standard is rich in information and is an excellent guide for the test planner.

### **Phase 1: Describe Unit Test Approach and Risks**

In this phase of unit testing planning the general approach to unit testing is outlined. The test planner:

- a) identifies test risks;
- b) describes techniques to be used for designing the test cases for the units;
- c) describes techniques to be used for data validation and recording of test results;
- d) describes the requirements for test harnesses and other software that interfaces with the units to be tested, for example, any special objects needed for testing object-oriented units.

During this phase the planner also identifies completeness requirements what will be covered by the unit test and to what degree (states, functionality, control, and data flow patterns). The planner also identifies termination conditions for the unit tests. Finally, the planner estimates resources needed for unit test, such as hardware, software, and staff, and develops a tentative schedule under the constraints identified at that time.

### **Phase 2: Identify Unit Features to be tested**

This phase requires information from the unit specification and detailed design description. The planner determines which features of each unit will be tested, for example: functions, performance requirements, states, and state transitions, control structures, messages, and data flow patterns. If some features will not be covered by the tests, they should be mentioned and the risks of not testing them be assessed. Input/output characteristics associated with each unit should

also be identified, such as variables with allowed ranges of values and performance at a certain level.

### **Phase 3: Add Levels of Detail to the Plan**

In this phase the planner refines the plan as produced in the previous two phases. The planner adds new details to the approach, resource, and scheduling portions of the unit test plan. As an example, existing test cases that can be reused for this project can be identified in this phase.

#### **3.1.2. Designing the unit tests**

Part of the preparation work for unit test involves unit test design. It is important to specify (i) the test cases (including input data, and expected outputs for each test case), and, (ii) the test procedures (steps required run the tests).

Test case data should be tabularized for ease of use, and reuse. To specifically support object-oriented test design and the organization of test data, Berard has described a test case specification notation. He arranges the components of a test case into a semantic network with parts, **Object\_ID**, **Test\_Case\_ID**, **Purpose**, and **List\_of\_Test\_Case\_Steps**. Each of these items has component parts. In the test design specification Berard also includes lists of relevant states, messages (calls to methods), exceptions, and interrupts. As part of the unit test design process, developers/testers should also describe the relationships between the tests. Test suites can be defined that bind related tests together as a group. All of this test design information is attached to the unit test plan. Test cases, test procedures, and test suites may be reused from past projects if the organization has been careful to store them so that they are easily retrievable and reusable.

## **2. The Test Harness**

In addition to developing the test cases, supporting code must be developed to exercise each unit and to connect it to the outside world. Since the tester is considering a stand-alone function/procedure/class, rather than a complete system, code will be needed to call the target unit, and also to represent modules that are called by the target unit. This code called the test harness, is developed especially for test and is in addition to the code that composes the system under development. The role of the test harness is shown in **Figure 7** and it is defined as follows:

The auxiliary code developed to support testing of units and components is called a test harness. The harness consists of drivers that call the target code and stubs that represent modules it calls.

The development of drivers and stubs requires testing resources. The drivers and stubs must be tested themselves to insure they are working properly and that they are reusable for subsequent releases of the software. Drivers and stubs can be developed at several levels of functionality. For example, a driver could have the following options and combinations of options:

call the target unit;

1. do 1, and pass inputs parameters from a table;
2. do 1, 2, and display parameters;
3. do 1, 2, 3 and display results (output parameters).



Figure 8:Test Harness

The stubs could also exhibit different levels of functionality. For example, a stub could:

1. display a message that it has been called by the target unit;
2. do 1, and display any input parameters passed from the target unit;
3. do 1, 2, and pass back a result from a table;
4. do 1, 2, 3, and display result from table.

### 3. Running the unit tests and recording results

Unit tests can begin when

- ❖ The units become available from the developers (an estimation of availability is part of the test plan),
- ❖ The test cases have been designed and reviewed, and
- ❖ The test harness, and any other supplemental supporting tools, are available.

The testers then proceed to run the tests and record results. The status of the test efforts for a unit, and a summary of the test results, could be recorded in a simple format such as shown in **Figure 9** . These forms can be included in the test summary report, and are of value at the weekly status meetings that are often used to monitor test progress. It is very important for the tester at any level of testing to carefully record, review, and check test results. The tester must determine from the results whether the unit has passed or failed the test. If the test is failed, the nature of the problem should be recorded in what is sometimes called a test incident report.

Other likely causes that need to be carefully investigated by the tester are the following:

- ❖ a fault in the test case specification (the input or the output was not specified correctly);
- ❖ a fault in test procedure execution (the test should be rerun);
- ❖ a fault in the test environment (perhaps a database was not set up properly);
- ❖ a fault in the unit design (the code correctly adheres to the design specification, but the latter is incorrect).

| Unit Test Worksheet    |                      |                    |           |
|------------------------|----------------------|--------------------|-----------|
| Unit Name: _____       |                      |                    |           |
| Unit Identifier: _____ |                      |                    |           |
| Tester: _____          |                      |                    |           |
| Date: _____            |                      |                    |           |
| Test case ID           | Status (run/not run) | Summary of results | Pass/fail |

Figure 9:Unit Test worksheet

The causes of the failure should be recorded in a test summary report, which is a summary of testing activities for all the units covered by the unit test plan.

### 2.6.2 Integration Testing

Integration test for procedural code has **two major goals**:

1. To detect defects that occur on the interfaces of units;
2. To assemble the individual units into working subsystems and finally a complete system that is ready for system test.

With a few minor exceptions, integration test should only be performed on units that have been reviewed and have successfully passed unit testing. A tester might believe erroneously that since a unit has already been tested during a unit test with a driver and stubs, it does not need to be retested in combination with other units during integration test. However, a unit tested in isolation may not have been tested adequately for the situation where it is combined with other modules.

Integration testing works best as an iterative process in procedural oriented system. One unit at a time is integrated into a set of previously integrated modules which have passed a set of integration tests. The interfaces and functionality of the new unit in combination with the previously integrated units is tested. When a subsystem is built from units integrated in this stepwise manner, then performance, security, and stress tests can be performed on this subsystem.

#### ❖ **Integration Test Planning**

Integration test must be planned. Planning can begin when high-level design is complete so that the system architecture is defined. Other documents relevant to integration test planning are the requirements document, the user manual, and usage scenarios. These documents contain structure charts, state charts, data dictionaries, cross-reference tables, module interface descriptions, data flow descriptions, messages and event descriptions, all necessary to plan integration tests. The strategy for integration should be defined. For procedural-oriented system the order of integration of the units of the units should be defined. This depends on the strategy selected. Consider the fact that the testing objectives are to assemble components into subsystems and to demonstrate that the subsystem functions properly with the integration test cases. For object-oriented systems a working definition of a cluster or similar construct must be described, and relevant test cases must be specified. In addition, testing resources and schedules for integration should be included in the test plan.

### **2.6.3 System Testing**

When integration tests are completed, a software system has been assembled and its major subsystems have been tested. At this point the developers/ testers begin to test it as a whole. System test planning should begin at the requirements phase with the development of a master test plan and requirements-based (black box) tests. System test planning is a complicated task. There are many components of the plan that need to be prepared such as test approaches, costs, schedules, test cases, and test procedures.

System testing itself requires a large number of resources. The goal is to ensure that the system performs according to its requirements. System test evaluates both functional behavior and quality requirements such as reliability, usability, performance and security. This phase of testing is especially useful for detecting external hardware and software interface defects, for example, those causing race conditions, and deadlocks, problems with interrupts and exception handling, and ineffective memory usage. After system test the software will be turned over to users for evaluation during acceptance test or alpha/beta test. The organization will want to be sure that the quality of the software has been measured and evaluated before users/clients are invited to use the system. In fact, system test serves as a good rehearsal scenario for acceptance test.

Because system test often requires many resources, special laboratory equipment, and long test times, it is usually performed by a team of testers. The best scenario is for the team to be part of an independent testing group. The team must do their best to find any weak areas in the software; therefore, it is best that no developers are directly involved.

There are several types of system tests as follows:

- ❖ **Functional testing**
- ❖ **Performance testing**
- ❖ **Stress testing**
- ❖ **Configuration testing**
- ❖ **Security testing**
- ❖ **Recovery testing**
- ❖ **Web Application Testing**

#### ❖ **Functional Testing**

System functional tests have a great deal of overlap with acceptance tests. Very often the same test sets can apply to both. Both are demonstrations of the system's functionality. Functional tests at the system level are used to ensure that the behavior of the system adheres to the requirements specification. All functional requirements for the system must be achievable by the system. For example, if a personal finance system is required to allow users to set up accounts, add, modify, and delete entries in the accounts, and print reports, the function-based system and acceptance tests must ensure that the system can perform these tasks. Clients and users will expect this at acceptance test time. Functional tests are black box in nature. The focus is on the inputs and proper outputs for each function. Improper and illegal inputs must also be handled by the

system. System behavior under the latter circumstances tests must be observed. All functions must be tested. In fact, the tests should focus on the following goals.

- ❖ All types or classes of legal inputs must be accepted by the software.
- ❖ All classes of illegal inputs must be rejected (however, the system should remain available).
- ❖ All possible classes of system output must exercise and examined.
- ❖ All effective system states and state transitions must be exercised and examined.
- ❖ All functions must be exercised.

#### ❖ **Performance Testing**

An examination of a requirements document shows that there are two major types of requirements:

**Functional requirements.** Users describe what functions the software should perform. We test for compliance of these requirements at the system level with the functional-based system tests.

**Quality requirements.** There are nonfunctional in nature but describe quality levels expected for the software. One example of a quality requirement is performance level. The users may have objectives for the software system in terms of memory use, response time, throughput, and delays. The goal of system performance tests is to see if the software meets the performance requirements. Testers also learn from performance test whether there are any hardware or software factors that impact on the system's performance. Performance testing allows testers to tune the system; that is, to optimize the allocation of system resources. For example, testers may find that they need to reallocate memory pools, or to modify the priority level of certain system operations. Testers may also be able to project the system's future performance levels. This is useful for planning subsequent releases.

#### ❖ **Stress Testing**

When a system is tested with a load that causes it to allocate its resources in maximum amounts, this is called stress testing. For example, if an operating system is required to handle 10 interrupts/second and the load causes 20 interrupts/second, the system is being stressed. The goal of stress test is to try to break the system; find the circumstances under which it will crash. This is sometimes called “**breaking the system**”. An everyday analogy can be found in the case where a suitcase being tested for strength and endurance is stomped on by a multiton elephant!

Stress testing is important from the user/client point of view. When systems operate correctly under conditions of stress then clients have confidence that the software can perform as required. Beizer suggests that devices used for monitoring stress situations provide users/clients with visible and tangible evidence that the system is being stressed.

### ❖ **Configuration Testing**

Typical software systems interact with hardware devices such as disc drives, tape drives, and printers. Many software systems also interact with multiple CPUs, some of which are redundant. Software that controls real-time processes, or embedded software also interfaces with devices, but these are very specialized hardware items such as missile launchers, and nuclear power device sensors.

In many cases, users require that devices be interchangeable, removable, or reconfigurable. For example, a printer of type X should be substitutable for a printer of type Y, CPU A should be removable from a system composed of several other CPUs, sensor A should be replaceable with sensor B. Very often the software will have a set of commands, or menus, that allows users to make these configuration changes. Configuration testing allows developers/testers to evaluate system performance and availability when hardware exchanges and reconfigurations occur.

**According to Beizer configuration testing has the following objectives:**

- ❖ Show that all the configuration changing commands and menus work properly.
- ❖ Show that all interchangeable devices are really interchangeable, and that they each enter the proper states for the specified conditions.
- ❖ Show that the systems' performance level is maintained when devices are interchanged, or when they fail.

Several types of operations should be performed during configuration test. Some sample operations for testers are:

- a) rotate and permute the positions of devices to ensure physical/ logical device permutations work for each device (e.g., if there are two printers A and B, exchange their positions);
- b) induce malfunctions in each device, to see if the system properly handles the malfunction;
- c) Induce multiple device malfunctions to see how the system reacts. These operations will help to reveal problems (defects) relating to hardware/software interactions when

hardware exchanges, and reconfigurations occur. Testers observe the consequences of these operations and determine whether the system can recover gracefully particularly in the case of a malfunction.

### ❖ **Security Testing**

Designing and testing software systems to ensure that they are safe and secure is a big issue facing software developers and test specialists. Recently, safety and security issues have taken on additional importance due to the proliferation of commercial applications for use on the Internet. If Internet users believe that their personal information is not secure and is available to those with intent to do harm, the future of e-commerce is in peril! Security testing evaluates system characteristics that relate to the availability, integrity, and confidentiality of system data and services. Users/clients should be encouraged to make sure their security needs are clearly known at requirements time, so that security issues can be addressed by designers and testers.

#### **Computer software and data can be compromised by:**

- a) Criminals intent on doing damage, stealing data and information, causing denial of service, invading privacy;
- b) Errors on the part of honest developers/maintainers who modify, destroy, or compromise data because of misinformation, misunderstandings, and/or lack of knowledge.

Both criminal behavior and errors that do damage can be perpetuated by those inside and outside of an organization. Attacks can be random or systematic. Damage can be done through various means such as:

- ❖ viruses;
- ❖ Trojan horses;
- ❖ trap doors;
- ❖ illicit channels.

The effects of security breaches could be extensive and can cause:

- ❖ loss of information;
- ❖ corruption of information;
- ❖ misinformation;
- ❖ privacy violations;

- ❖ denial of service.

A password checker can enforce any rules the designers deem necessary to meet security requirements.

Password checking and examples of other areas to focus on during security testing are described below.

- ❖ **Password Checking:** Test the password checker to ensure that users will select a password that meets the conditions described in the password checker specification. Equivalence class partitioning and boundary value analysis based on the rules and conditions that specify a valid password can be used to design the tests.
- ❖ **Legal and Illegal Entry with Passwords:** Test for legal and illegal system/data access via legal and illegal passwords.
- ❖ **Password Expiration:** If it is decided that passwords will expire after a certain time period, tests should be designed to insure the expiration period is properly supported and that users can enter a new and appropriate password.
- ❖ **Encryption:** Design test cases to evaluate the correctness of both encryption and decryption algorithms for systems where data/messages are encoded.
- ❖ **Browsing:** Evaluate browsing privileges to ensure that unauthorized browsing does not occur. Testers should attempt to browse illegally and observe system responses. They should determine what types of private information can be inferred by both legal and illegal browsing.
- ❖ **Trap Doors:** Identify any unprotected entries into the system that may allow access through unexpected channels (trap doors). Design tests that attempt to gain illegal entry and observe results. Testers will need the support of designers and developers for this task. In many cases an external “tiger team” as described below is hired to attempt such a break into the system.
- ❖ **Viruses:** Design tests to ensure that system virus checkers prevent or curtail entry of viruses into the system. Testers may attempt to infect the system with various viruses and observe the system response. If a virus does penetrate the system, testers will want to determine what has been damaged and to what extent.
- ❖ **Recovery Testing**

Recovery testing subjects a system to losses of resources in order to determine if it can recover properly from these losses. This type of testing is especially important for transaction systems, for example, on-line banking software.

**Beizer advises that testers focus on the following areas during recovery testing:**

1. **Restart.** The current system state and transaction states are discarded. The most recent checkpoint record is retrieved and the system initialized to the states in the checkpoint record. Testers must insure that all transactions have been reconstructed correctly and that all devices are in the proper state. The system should then be able to begin to process new transactions.
2. **Switchover.** The ability of the system to switch to a new processor must be tested. Switchover is the result of a command or a detection of a faulty processor by a monitor. In each of these testing situations all transactions and processes must be carefully examined to detect:
  - a) loss of transactions;
  - b) merging of transactions;
  - c) incorrect transactions;
  - d) An unnecessary duplication of a transaction.

A good way to expose such problems is to perform recovery testing under a stressful load. Transaction inaccuracies and system crashes are likely to occur with the result that defects and design flaws will be revealed.

### ❖ **Regression Testing**

Regression testing is not a level of testing, but it is the retesting of software that occurs when changes are made to ensure that the new version of the software has retained the capabilities of the old version and that no new defects have been introduced due to the changes. Regression testing can occur at any level of test, for example, when unit tests are run the unit may pass a number of these tests until one of the tests does reveal a defect. The unit is repaired and then retested with all the old test cases to ensure that the changes have not affected its functionality.

Regression tests are especially important when multiple software releases are developed. Users want new capabilities in the latest releases, but still expect the older capabilities to remain in place. This is where regression testing plays a role. Test cases, test procedures, and other test-related items from previous releases should be available so that these tests can be run with the

new versions of the software. Automated testing tools support testers with this very time-consuming task.

### ❖ Web Application Testing

Web testing is the name given to software testing that focuses on web applications [2]. Web application testing is a mandatory skill for software test engineers these days. It checks bottlenecks and performance leakage in the software or web application to be tested.

Web application testing is a quality assurance activity evaluating the system under test (SUT) by observing its execution to reveal failures [3]. When the SUT's external behavior differs from what we expect of the SUT according to its requirements or some other description of the expected behavior, a failure is discovered [4].

### **Type of Web Application Testing**

Web application testing can be divided into two main categories, manual testing, and automated web application testing [5]. Both categories have their strengths and weaknesses.

#### **A. Manual Testing**

Manual testing is a testing process that is carried out manually to find defects without the usage of tools or automation scripting. A test plan document is created by the test lead which describes the detailed and systematic approach to testing a software application and which the tester has to be followed. Typically, the test plan includes a complete understanding of what the ultimate workflow will be. To ensure the completeness of testing, test cases or test scenarios are created [5].

The tester runs the test cases manually to see whether they have any defects. In manual testing, the tester has to pretend as an application user and use all kinds of functions available to find out its right behavior [6]. Manual testing can't be used again, no facility for scripting, and few errors may remain undetected. In addition, manually performed software testing is a time-consuming and burdensome task. Therefore, authors have developed various automated methods and tools to help developers test the quality of a web application.

#### **B. Automated Testing**

Automated testing focuses on replacing manual human activities with systems or devices that enhance the efficiency of the software testing process. It uses tools, scripts, and software to perform

test cases by repeating pre-defined actions. It is suitable for regression testing, performance testing, load testing, or highly repeatable functional test cases.

The test case is a crucial component of the testing process. A test case outlines the conditions under which SUT detects a failure. When a test case reveals a failure, it is successful. A test case embodies the input values needed to execute the SUT [4]. Therefore, test case inputs vary in nature, ranging from user inputs to method calls with the test-case values as parameters. To evaluate the results of test cases, testers must first know where an element is situated, then conduct an action on the web element and examine the result.

### **Test Script**

Automated is one of the more popular and available strategies to reduce testing effort. It develops test scripts that will be used later to execute test cases instead of human [7]. Test script is a set of instructions that are performed on a SUT to verify that the system performs as expected. Each test script command is composed of locator, test action, and test data. The test action specifies the action performed on that specific web elements [8]. A test script is part of a test case, and a test case can have either a single test script or multiple test scripts. The script code will typically do the following one or more times. Identify input elements in the user interface (UI), navigate to the required UI component, identify output element, and verify that output elements display the result, read the result from the output element, assert that output value is equal to the expected value, and write the result of the test to a log [9].

A test automation scripting technique is a set of instructions to define the method of testing to be used by the quality assurance team to get the pre-determined or expected test results. It is important to note that test scripting techniques are not mandatory for test case execution, but they help test teams conduct test case executions with higher accuracy and efficiency. The following are the most often used scripting techniques for test automation by quality assurance teams.

#### **A. Linear Scripting Techniques**

The linear scripting approach is a simple record and playback used by a test engineer to automate test case of a system. John Kent [10] explains the idea behind the linear technique, which is simply to set the test tool to the record mode while performing actions on the SUT. The generated recorded script consists of a series of testing instructions using the programming language supported by the tool. Linear scripting technique is easy to use and does not need to have any programming skills.

The main disadvantage is the recorded script can only work under the same conditions as when it was recorded the first time. If a simple error happened or unexpected normal events during a test run, it will not be handled correctly by the test script, and it is not reliable enough, even if the application has not changed [11]. One of the major limitations of record and replay tools are if the script author who needed to repeat an action many times would need to record that action and maintain each one of the actions individually.

### **B. Structural and Shared Scripting Techniques**

Structured scripting technique uses structured programming instructions, which either be control structures or calling structures [11]. It uses control structures in the scripts. These control structures enable testers to control the flow of the test script. The control structures are typical ‘if-else’, ‘switch’, ‘for’, ‘while’ statements that help in implementing decision making in the script, to perform some tasks iteratively and capacity to call other common functions that house commonly needed functionality [12]. The most important advantage of the structured technique is that the test script can validate for specific conditions to determine if the executed test passed or failed according to these conditions. However, it requires testing skills as well as programming skills.

### **C. Data-Driven Scripting Techniques**

This technique separates data from scripts and stores it as files on a disk in an external repository. As a result, the test script contains the programmed code. It may need this when the data needs to be varied over the test run.

One benefit of this approach is that it makes it possible to modify a test without actually having to change the test case's code. This makes it possible to separate responsibility for creating tests between test engineers who have coding skills and those who don't. Those with coding skills can create test scripts and encode the test logic in them, and test engineers who don't have coding skills can create and edit the test data that the test scripts use to test the AUT.

### **D. Keyword-Driven Scripting Techniques**

It is an approach in which the control to check and execute operations is maintained with external data files by dividing all test steps by using reusable keyword functions. So, the test data and the operations or sequence of the test are planned in an external data file and an extra library is needed to interpret this data in addition to the conventional script. It is an extension of data-driven testing. A tester can create test cases by using Microsoft excel using a variety of keywords (*e.g.*, click-element, send-keys verify-element, and store text). As the test gets executed and processes the

excel file in the driver script one by one against the keywords present in the excel file [13]. The advantage of this scripting technique is to add new test cases and change to test cases requires no code change. The main disadvantage is that webpages change an element identification or dynamism of web applications to become an issue.

#### **2.6.4 Acceptance Testing**

Acceptance testing is the last phase of testing and is used to assess whether or not the final piece of software is ready for delivery. It involves ensuring that the product is in compliance with all of the original business criteria and that it meets the end user's needs. This requires the product be tested both internally and externally, meaning you'll need to get it into the hands of your end users for beta testing along with those of your QA team. Beta testing is key to getting real feedback from potential customers and can address any final usability concerns. When software is being developed for a specific client, acceptance tests are carried out after system testing. The acceptance tests must be planned carefully with input from the client/users.

Acceptance test cases are based on requirements. The user manual is an additional source for test cases. System test cases may be reused. The software must run under real-world conditions on operational hardware and software. The software-under-test should be stressed. For continuous systems the software should be run at least through a 25-hour test cycle. Conditions should be typical for a working day. Typical inputs and illegal inputs should be used and all major functions should be exercised. If the entire suite of tests cannot be run for any reason, then the full set of tests needs to be rerun from the start.

**Acceptance tests** are a very important milestone for the developers. At this time the clients will determine if the software meets their requirements. Contractual obligations can be satisfied if the client is satisfied with the software. Development organizations will often receive their final payment when acceptance tests have been passed.

Acceptance testing can be divided into two, i.e. **Beta testing** and **Alpha testing**. Beta User testing of a completed information system using real data in the real user environment. Beta version of the software is released to a limited number of end-users of the product to obtain feedback on the product quality. Beta testing reduces product failure risks and provides increased quality of the product through customer validation. Beta testing is performed by Clients or End Users who are not employees of the organization. Alpha testing is a type of user acceptance

testing or internal acceptance testing conducted by the testing team at the developer's site. This is done to ensure that the application's business requirements are met before releasing it to the client or the end-user. It is carried out in the later phase of software development using both black-box and white-box testing techniques.

After acceptance testing the client will point out to the developers which requirement have/have not been satisfied. Some requirements may be deleted, modified, or added due to changing needs. If the client has been involved in prototype evaluations, then the changes may be less extensive. If the client is satisfied that the software is usable and reliable, and they give their approval, then the next step is to install the system at the client's site. If the client's site conditions are different from that of the developers, the developers must set up the system so that it can interface with client software and hardware. Retesting may have to be done to ensure that the software works as required in the client's environment. Table 1 shows the conclusion of level of software testing.

Table 5: Level of Software Testing Conclusion

|       | Unit Testing                              | Integration                                                           | System                                            | Acceptance                                         |
|-------|-------------------------------------------|-----------------------------------------------------------------------|---------------------------------------------------|----------------------------------------------------|
| Why   | To ensure the code is developed correctly | To ensure the ties between the system component functions as required | To ensure the whole system worked when integrated | To ensure customers and end user expectation met   |
| Who   | Developer/Technical Architects            | Developer/Technical Architects                                        | SDET/Manual QA, BA/ Product owner                 | Developer, SDET/Manual QA,/Product owner, end user |
| What  | All new code                              | New web service components, controllers                               | Scenario testing<br>User flow                     | Verifying Acceptance testing                       |
| When  | As soon as a new code is written          | As soon as a new component is written                                 | When the product is complete                      | When the product is shipped                        |
| Where | Local Dev+ Continuous integration         | Local Dev+ CI (Part of the build)                                     | Staging Environment                               | CI/ Test Environment                               |
| How   | Automated, Junit, TestNG                  | Automated, Soup UI                                                    | Automated (web driver)                            | Automated(Cucumber)                                |

## Review Questions

1. Which of the following should **not** normally be the direct objective for software testing?
  - A. To find faults in the software
  - B. To supply records of software errors to be used for error prevention
  - C. To assess whether the software is ready to release
  - D. To demonstrate that the software doesn't work
2. When does integration testing conducted?
  - A. As soon as a new code is written
  - B. When a new component is written
  - C. When the product is complete
  - D. When the product is shipped
3. When should testing be stopped?
  - A. Bug rate falls below a certain level and no high priority bugs are identified
  - B. Deadline (Testing Release)
  - C. Completion of Functional and code coverage to a certain point
  - D. All
4. Which one of the following statements correctly describes the key principles of software testing?
  - A. By using automated testing, it is possible to test everything.
  - B. With sufficient effort and tool support, exhaustive testing is feasible for all software.
  - C. It is impossible to test all input and precondition combinations in a system
  - D. The purpose of testing is to prove the absence of a defect

## Chapter Three: Software Testing Techniques

### Objectives

*At the end of this chapter students will be able to*

- ☞ understand and describe the basic concepts of functional (black box) software testing.
- ☞ List a range of different software testing techniques and strategies and be able to apply specific method to the projects
- ☞ variety of testing techniques that can be used in both white-box and black-box testing strategies
- ☞ Execute specific software tests with well-defined objectives and targets

### 3.1 Classification of software Testing Techniques

Testing strategies define the approaches for designing test cases. They can be responsibility-based (also known as a black box), an implementation-based, or hybrid (also known as a grey box). Classification based on the criterion to measure the adequacy of a set of test cases:

- ❖ **Coverage-based testing:** How many statements or requirements have been tested
- ❖ **Fault-based testing:** how many seeded faults are found
- ❖ **Error-based testing:** focus on error-prone points

#### 3.1.1 White Box Strategies

White box strategies design test cases based on a coded representation of the component under test and of a coverage model that specifies the parts of the representation that must be exercised by a test suite. As an example, in the case of traditional software the control flow graph is a typical test model, while statement coverage, branch coverage, or basis-path coverage are possible code coverage models [14].

#### White-box Testing Techniques

##### A. Control-flow-based testing

Control flow testing is a testing technique that comes under white box testing. The aim of this technique is to determine the execution order of statements or instructions of the program through a control structure. The control structure of a program is used to develop a test case for the program. In this technique, a particular part of a large program is selected by the tester to set the testing path. It is mostly used in unit testing. Test cases represented by the control graph of the program.

**Control Flow Graph** is formed from the node, edge, decision node, junction node to specify all possible execution path.

**Nodes** in the control flow graph are used to create a path of procedures. Basically, it represents the sequence of procedures which procedure is next to come so, the tester can determine the sequence of occurrence of procedures.

We can see below in example the first node represents the start procedure and the next procedure is to assign the value of n after assigning the value there is decision node to decide next node of procedure as per the value of n if it is 18 or more than 18 so Eligible procedure will execute

otherwise if it is less than 18 Not Eligible procedure executes. The next node is the junction node, and the last node is stop node to stop the procedure.

## Edge

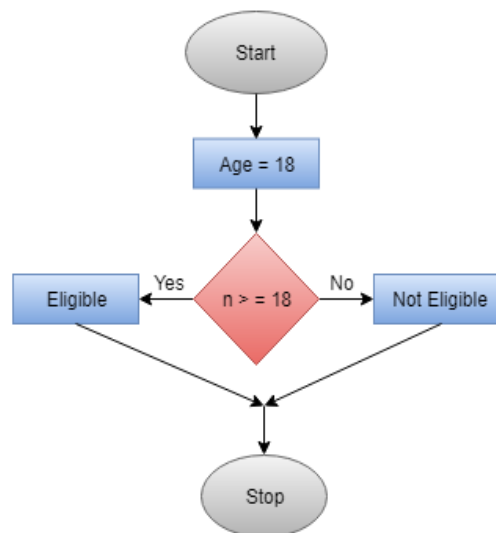
Edge in control flow graph is used to link the direction of nodes. We can see below in example all arrows are used to link the nodes in an appropriate direction.

## Decision node

Decision node in the control flow graph is used to decide next node of procedure as per the value. We can see below in example decision node decide next node of procedure as per the value of n if it is 18 or more than 18 so Eligible procedure will execute otherwise if it is less than 18, Not Eligible procedure executes. In the control flow graph, start, age, eligible, not eligible and stop are the nodes,  $n \geq 18$  is a decision node to decide which part (if or else) will execute as per the given value. Connectivity of the eligible node and not eligible node is there on the stop node.

Test cases are designed through the flow graph of the programs to determine the execution path is correct or not. All nodes, junction, edges, and decision are the essential parts to design test cases.

```
public class VoteEligibilityAge{  
  
    public static void main(String []args){  
        int n=45;  
        if(n>=18)  
        {  
            System.out.println("You are eligible for voting");  
        } else  
        {  
            System.out.println("You are not eligible for voting");  
        }  
    }  
}
```



## B. Statement coverage

Statement coverage is one of the widely used software testing. It comes under white box testing. Statement coverage technique is used to design white box test cases. This technique involves execution of all statements of the source code at least once. It is used to calculate the total number of executed statements in the source code out of total statements present in the source code.

Statement coverage derives scenario of test cases under the white box testing process which is based upon the structure of the code.

$$\text{Statement coverage} = \frac{\text{Number of executed statements}}{\text{Total number of statements}} * 100$$

Example:

```
Prints (int a, int b) {
    int result = a+ b;
    If (result> 0)
        Print ("Positive", result)
    Else
        Print ("Negative", result)
}
```

----- Printsum is a function

----- End of the source code

**Scenario 1:** If A=3, B=9

```
1 Prints (int a, int b) {
2   int result = a+ b;
3   If (result> 0)
4     Print ("Positive", result)
5   Else
6     Print ("Negative", result)
7 }
```

The statements marked in yellow color are those which are executed as per the scenario

Number of executed statements = 5, Total number of statements = 7

Statement Coverage: 5/7 = 71%



**Scenario 2:** If A = -3, B = -9

```
1 Prints (int a, int b) {
2   int result = a+ b;
3   If (result> 0)
4     Print ("Positive", result)
5   Else
6     Print ("Negative", result)
7 }
```

The statements marked in yellow color are those which are executed as per the scenario.

Number of executed statements = 6

Total number of statements = 7

$$\text{Statement Coverage} = \frac{\text{Number of executed statments}}{\text{Total number of statments}}$$

Statement Coverage: 6/7 = 85%

### C. Branch testing

Branch coverage technique is used to cover all branches of the control flow graph. It covers all the possible outcomes (true and false) of each condition of decision point at least once. Branch coverage technique that ensures that every branch of each decision point must be executed.

However, branch coverage technique and decision coverage technique are very similar, but there is a key difference between the two. Decision coverage technique covers all branches of each decision point whereas branch testing covers all branches of every decision point of the code.

In other words, branch coverage follows decision point and branch coverage edges. Many different metrics can be used to find branch coverage and decision coverage, but some of the most basic metrics are: finding the percentage of program and paths of execution during the execution of the program.

#### 3.1.2 Black Box Strategies

Black box techniques do not require knowledge of software implementation items under test since test cases are designed based on an item's specified or expected functionality. One main issue with black-box testing of Web applications is the choice of a suitable model for specifying the behavior of the application to be tested and to derive test cases. Indeed, this behavior may significantly depend on the state of data managed by the application and on user input, with the consequence of a state explosion problem even in the presence of applications implementing a few simple requirements.

#### ❖ Black Box Testing Techniques

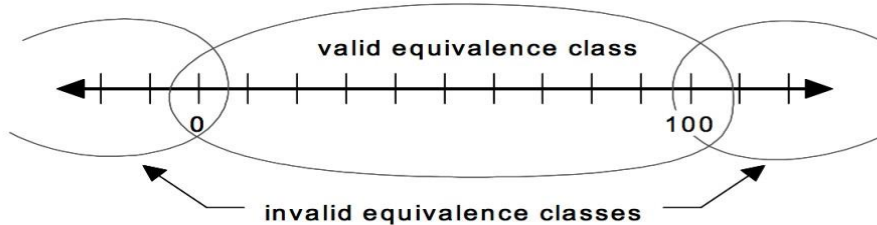
##### A. Equivalence class Testing

A black-box testing method that divides the input domain of a program into classes of data from which test cases are derived. Test case design is based on an evaluation of equivalence classes for an input condition. An equivalence class represents a set of valid or invalid states for input conditions. From each equivalence class, test cases are selected so that the largest number of attributes of an equivalence class is exercised at once

Equivalence testing divides the space of all possible inputs into equivalence groups such that the program “**behaves the same**” on each group

## Two steps:

1. Partitioning the values of input parameters into equivalence groups
2. Choosing the test input values



## Guidelines for Defining Equivalence Classes

- ❖ If an input condition specifies **a range**, one valid and two invalid equivalence classes are defined
  - Input range: 1 – 10    Eq classes: {1..10}, {x < 1}, {x > 10}
- ❖ If an input condition requires **a specific value**, one valid and two invalid equivalence classes are defined
  - Input value: 250    Eq classes: {250}, {x < 250}, {x > 250}
- ❖ If an input condition specifies **a member of a set**, one valid and one invalid equivalence class are defined
  - Input set: {-2.5, 7.3, 8.4}    Eq classes: {-2.5, 7.3, 8.4}, {any other x}
- ❖ If an input condition is **a Boolean value**, one valid and one invalid class are defined
  - Input: {true condition}    Eq classes: {true condition}, {false condition}

## B. Boundary Value Testing

Boundary value analysis, BVA, tests the behavior of a program at the boundaries. When checking a range of values, after selecting the set of data that lie in the valid partitions, next is to check how the program behaves at the boundary values of the valid partitions. Boundary value analysis is most common when checking a range of numbers.

For each range, there are two boundaries, the lower boundary (start of the range) and the upper boundary (end of the range) and the boundaries are the beginning and end of each valid partition.

We should design test cases which exercises the program functionality at the boundaries, and with values just inside and just outside the boundaries.

Here, the assumption is that if the program works correctly for these extreme cases, then it will work correctly for all values in between the valid partition. Testing has shown that defects that arise when checking a range of values, the most defects are near or at the boundaries.

### *Boundary Value Analysis Example*

A program which accepts an integer in the range **-100** to **+100**, there would be three sets of valid equivalent partitions: these are -10 to -1, the negative range 0, Zero and 1 to 10, the positive range. For each range, there are minimum and maximum values at each boundary. For the negative range the lower boundary is -10 and the upper boundary is -1. At each boundary, three conditions should be checked.

-101, **-100**, -99 ... .. -2, **-1**,

0, -1, **0**, +1

0, **1**, 2 ... .. 99, **100**, 101

We might have noticed that by selecting values at the boundaries for each partition there are some values that overlap. That is, they would appear in our test conditions when we check the boundaries. We should of course make redundant those values that overlap to eliminate unnecessary test cases.

Another worthy note to consider is that because the range goes from -100 to +100, then effectively, the “boundary values” -2, -1 and -99 are considered as equivalent. That is, the behavior of the system is the same (or should be the same) when testing with values -99, -2 and -1.

However, because -99 is close to the upper limit boundary, and is most likely to reveal defects, we keep the value -99 in our test case and scrap conditions for -2 and -1. The same applies to the positive range boundaries, that is, +1, +2 and +99 are expected to give the same behavior. However, because 99 is close to the upper range boundary, we keep 99 in our list of data to test and get rid of values 1 and 2.

Therefore, our list of data to check at the boundaries become

-101, **-100**, -99 ... .. 0 ... .. 99, **100**, 101

If our range of data was from 0 to +10 then the boundary value analysis would give us the following values to test:

-1, **0**, 1 ... .. 9, **10**, 11

Here, 1 and 9 are both part of the test condition because each are at the boundary of the whole range and can reveal defects.

It is important to note that that boundary values analysis can be applied to float numbers as well as integers. The only difference is that when analyzing boundaries for float numbers, we should test to the closest decimal point.

Example, if we have a range from 5.5 to 9.9, then the set of data at boundaries becomes

5.4, **5.5**, 5.6 ... .. 9.8, **9.9**, 10.0

### **C. Error Guessing**

Error guessing is a testing technique that makes use of a tester's skill, intuition and experience in testing similar applications to identify defects that may not be easy to capture by the more formal techniques. It is usually done after more formal techniques are completed.

### **D. Decision Table Testing**

Decision table technique is one of the widely used case design techniques for black box testing. This is a systematic approach where various input combinations and their respective system behavior are captured in a tabular form. That's why it is also known as a cause-effect table. This technique is used to pick the test cases in a systematic manner; it saves the testing time and gives good coverage to the testing area of the software application.

Decision table technique is appropriate for the functions that have a logical relationship between two and more than two inputs. This technique is related to the correct combination of inputs and determines the result of various combinations of input. To design the test cases by decision table technique, we need to consider conditions as input and actions as output.

when you want to use an email account, for this you need to enter the email and its associated password. If both email and password are correctly matched, the user will be directed to the email account's homepage; otherwise, it will come back to the login page with an error message specified with "Incorrect Email" or "Incorrect Password."

Now, let's see how a decision table is created for the login function in which we can log in by using email and password. Both the email and the password are the conditions, and the expected result is action.

Table 6: Decision Table Example

|                                 |              |                    |                 |                 |
|---------------------------------|--------------|--------------------|-----------------|-----------------|
| <b>Email (condition1)</b>       | T            | T                  | F               | F               |
| <b>Password (condition2)</b>    | T            | F                  | T               | F               |
| <b>Expected Result (Action)</b> | Account Page | Incorrect password | Incorrect email | Incorrect email |

In the table, there are four conditions or test cases to test the login function. In the first condition if both email and password are correct, then the user should be directed to account's Homepage.

In the second condition if the email is correct, but the password is incorrect then the function should display Incorrect Password. In the third condition if the email is incorrect, but the password is correct, then it should display Incorrect Email. Now, in fourth and last condition both email and password are incorrect then the function should display Incorrect Email.

In this example, all possible conditions or test cases have been included, and in the same way, the testing team also includes all possible test cases so that upcoming bugs can be cured at testing level. In order to find the number of all possible conditions, tester uses  $2^n$  formula where n denotes the number of inputs; in the example there is the number of inputs is 2 (one is true and second is false).

**Number of possible conditions =  $2^{\text{Number of Values of the second condition}}$**   
**Number of possible conditions =  $2^2 = 4$**

### 3.1.3 Grey Box Strategies

Grey box testing strategies combine black box and white box testing approaches to design test cases: they aim at testing a piece of software against its specification but using some knowledge of its internal workings.

## Review Questions

1. For registration use case in the User Name field the system accepts 8 to 15 alpha characters. Using BVA technique \_\_\_\_\_ will be the possible input values for the test case.
  - A. 8,9,15,16
  - B. 7,8,14,15
  - C. 8,9,10,15,16,17
  - D. 7,8,9,14,15,16
5. Which one of the following is not true about controlling activities in software testing process?
  - A. Balances appropriate level between too much and too little testing activities
  - B. Suspend the testing process if testing delay based on the schedule
  - C. Guiding a course of action to meet objectives of software testing
  - D. Establish a monitoring and reporting system
6. Which of the following statements is not correct?
  - A. A test set achieves 100% branch coverage will also achieve 100% statement coverage.
  - B. A minimal test set that achieves 100% branch coverage will generally detect more faults than one that achieves 100% statement coverage.
  - C. A minimal test set that achieves 100% statement coverage will generally detect more faults than one that achieves 100% branch coverage
  - D. None
7. Explain the following Black-box testing techniques in detail using real-world scenarios as an example.
  - A. Combinatorial Testing
  - B. Grammar-Based Testing
  - C. Module Base Testing
  - D. Orthogonal matrix
8. Explain the following White-box testing techniques in detail using real-world scenarios as an example.
  - A. Data-flow-based testing
  - B. Loop testing
  - C. Path coverage

## Chapter Four: Test Management

### Objectives

*At the end of this chapter students will be able to*

- ☞ Manage a testing project by implementing the mission, goals and testing processes established for the testing organization
- ☞ Organize and lead risk identification and risk analysis sessions and use the results of such sessions for test estimation, planning, monitoring and control
- ☞ Create and implement test plans consistent with organizational policies and test strategies
- ☞ Continuously monitor and control the test activities to achieve project objectives
- ☞ Assess and report relevant and timely test status to project stakeholders
- ☞ Have an ability to use various communication methods and skills to communicate with their teammates to conduct their practice-oriented software testing projects.
- ☞ To learn how to planning a test project, design test cases and data, conduct testing operations, manage software problems and defects, generate a testing report
- ☞ To learn how to write software testing documents, and communicate with engineers in various forms
- ☞ Have an ability understand and identify various software testing problems, and solve these problems by designing and selecting software test models, criteria, strategies, and methods.

#### **4.1 Test Management**

Test management, process of managing the tests. A test management is also performed using tools to manage both types of tests, automated and manual, that have been previously specified by a test procedure.

Test management tools allow automatic generation of the requirement test matrix (RTM), which is an indication of functional coverage of the application under test (SUT).

Test Management tool often has multifunctional capabilities such as testware management, test scheduling, the logging of results, test tracking, incident management and test reporting.

#### **4.2. Test Management Responsibilities**

Test Management has a clear set of roles and responsibilities for improving the quality of the product. It helps the development and maintenance of product metrics during the course of project. Test management enables developers to make sure that there are fewer design or coding faults.

#### **4.3. Profile of Test Personnel**

Testing requires the persons with wide variety of skills and communication.

The roles in test process are:

##### **❖ Test Manager**

**A Test Manager is most commonly responsible for the following activities:**

- ✓ Able to identify how the test teams align within an organization and how this team would achieve the roadmap identified for the project and the organization.
- ✓ He needs to identify the scope of the testing required for a particular release based on the requirements of the document
- ✓ Put out the Test Plan after discussions with the test team and have it reviewed and approved by the Management/ Development team
- ✓ Figure out what skills are required and balance the test resources accordingly with those needs based on their own interests as well.
- ✓ Identify the tools for Test Reporting, Test Management, Test Automation, etc.

### ❖ **Test Lead**

The test lead assists the test manager and works with a team of test engineers on individual projects. A Test Lead is most commonly responsible for the following activities:

- ✓ Develop test plan and the guidelines to create test conditions, test cases, expected results and execution scripts.
- ✓ Provide guidelines on how to manage defects.
- ✓ Communicate to the test team any changes that need to be made to the test deliverables or application and when they will be completed.
- ✓ Identify the tools for Test Reporting, Test Management, Test Automation, etc.
- ✓ Facilitate defect communications between testing team and technical / development team

### ❖ **Test Designer**

#### **Test Designer Responsibilities**

Designs the necessary test cases and sets up the order of execution.

- ✓ Development and testing knowledge
- ✓ Knowledge on specification-based techniques
- ✓ Knowledge on functional requirements

### ❖ **Test Automation Engineer**

Evaluate the possibility and necessity of the test automation

- ✓ Experience as a tester
- ✓ Programming skills
- ✓ Excellent skills in operating the applied tools

### ❖ **System Administrator**

Set up the test environment so that it meets the system requirement

- ✓ System administration experience
- ✓ Knowledge on development and testing tools
- ✓ Installation and operations of various system software

### ❖ Testing Engineer

Execute tests in accordance with the test case specifications

- ✓ Basic knowledge of software and testing
- ✓ Operation of testing tools
- ✓ Experience in conducting test

### ❖ Technical Expert

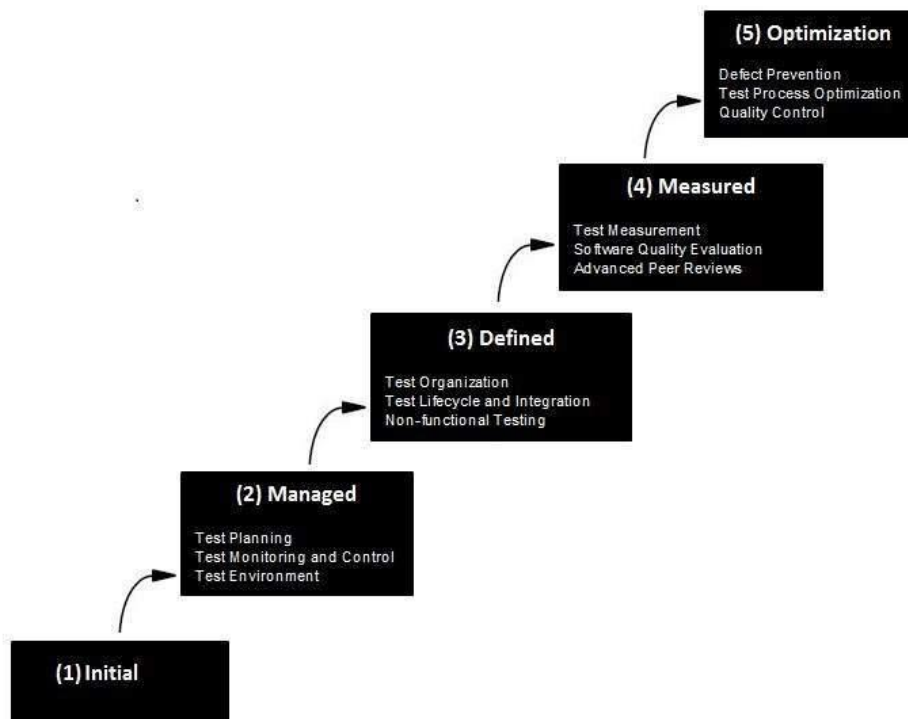
Support the test team where necessary

- ✓ Database administration
- ✓ Networking experts
- ✓ Security experts

## 4.4 Test Maturity Model

Test maturity model is based on capability maturity model specifies an increasing series of levels of a software development organization. The higher the level, the better the software development process, hence reaching each level is an expensive and time-consuming process.

### Level of CMM



**Level One: Initial** - The software process is characterized as inconsistent, and occasionally even chaotic. Defined processes and standard practices that exist are abandoned during a crisis. Success of the organization majorly depends on an individual effort, talent, and heroics. The heroes eventually move on to other organizations taking their wealth of knowledge or lessons learnt with them.

**Level Two: Repeatable** - This level of Software Development Organization has a basic and consistent project management processes to track cost, schedule, and functionality. The process is in place to repeat the earlier successes on projects with similar applications. Program management is a key characteristic of a level two organization.

**Level Three: Defined** - The software process for both management and engineering activities are documented, standardized, and integrated into a standard software process for the entire organization and all projects across the organization uses an approved, tailored version of the organization's standard software process for developing, testing and maintaining the application.

**Level Four: Managed** - Management can effectively control the software development effort using precise measurements. At this level, organization set a quantitative quality goal for both software process and software maintenance. At this maturity level the performance of processes is controlled using statistical and other quantitative techniques, and is quantitatively predictable.

**Level Five: Optimizing** - The Key characteristic of this level is focusing on continually improving process performance through both incremental and innovative technological improvements. At this level, changes to the process are to improve the process performance and at the same time maintaining statistical probability to achieve the established quantitative process-improvement objectives.

#### **4.5 Test Plan**

Test planning, the most important activity to ensure that there is initially a list of tasks and milestones in a baseline plan to track the progress of the project. It also defines the size of the test effort. It is the main document often called as master test plan or a project test plan and usually developed during the early phase of the project.

### **4.5.1 Test Plan Components**

#### **1. Test Plan Identifier**

Each test plan should have a unique identifier so that it can be associated with a specific project and become a part of the project history. The project history and all project-related items should be stored in a project database or come under the control of a configuration management system. Tester dependent on CM because we know the exact version of the test and the test object.

#### **2. Introduction**

In this section the test planner gives an overall description of the project, the software system being tested or maintained.

- ❖ Purpose
- ❖ Project overview
- ❖ Audience

#### **9. Item to be Tested**

This is a listing of the entities to be tested and should include names, identifiers, and version/revision numbers for each entity. The items listed could include procedures, classes, modules, subsystems, and systems.

#### **10. Features to Be Tested**

Features may be described as distinguishing characteristics of a software component or system.

#### **11. Approach**

This section of the test plan provides broad coverage of the issues to be addressed when testing the target software. Testing activities are described. The level of descriptive detail should be sufficient so that the major testing tasks and task durations can be identified.

#### **12. Item Pass/Fail Criteria**

Given a test item and a test case, the tester must have a set of criteria to decide on whether the test has been passed or failed upon execution.

### **13. Suspension and Resumption Criteria**

In this section of the test plan, criteria to suspend and resume testing are described. In the simplest of cases testing is suspended at the end of a working day and resumed the following morning. For some test items this condition may not apply and additional details need to be provided by the test planner. The test plan should also specify conditions to suspend testing based on the effects or criticality level of the failures/defects observed. Conditions for resuming the test after there has been a suspension should also be specified. For some test items resumption may require certain tests to be repeated.

### **14. Test Deliverables**

Execution-based testing has a set of deliverables that includes the test plan along with its associated test design specifications, test procedures, and test cases. The latter describe the actual test inputs and expected outputs. Deliverables may also include other documents that result from testing such as test logs, test transmittal reports, test incident reports, and a test summary report.

### **15. Testing Tasks**

In this section the test planner should identify all testing-related tasks and their dependencies. Using a Work Breakdown Structure (WBS) is useful here.

### **16. The Testing Environment**

Here the test planner describes the software and hardware needs for the testing effort. For example, any special equipment or hardware needed such as emulators, telecommunication equipment, or other devices should be noted. The planner also needs to specify any special software needs such as coverage tools, databases, and test data generators. Security requirements for the testing environment may also need to be described.

### **17. Responsibilities**

The staff that will be responsible for test-related tasks should be identified. This includes personnel who will be:

- ❖ Transmitting the software-under-test
- ❖ Developing test design specifications, and test cases

- ❖ Executing the tests and recording results
- ❖ Tracking and monitoring the test efforts
- ❖ Checking results
- ❖ Interacting with developers
- ❖ Managing and providing equipment
- ❖ Interacting with the users/customers

## **18. Staffing and Training Needs**

The test planner should describe the staff and the skill levels needed to carry out test-related responsibilities such as those listed in the section above. Any special training required to perform a task should be noted.

## **19. Scheduling**

Task durations should be established and recorded with the aid of a task networking tool. A tester will find that PERT and Gantt charts are very useful tools. Test milestones should be established, recorded, and scheduled. These milestones usually appear in the project plan as well as the test plan. Schedules for use of staff, tools, and equipment, should also be specified.

## **20. Test Effort Estimation**

Lists out all the activities that have to be performed by the QA team and estimates how many man-hours each activity is going to take. The IEEE standard for test plan documentation does not include a separate cost component in its specification of a test plan. This is the usual case for many test plans since very often test costs are allocated in the overall project management plan. The project manager in consultation with developers and testers estimates the costs of testing. If the test plan is an independent document prepared by the testing group and has a cost component, the test planner will need tools and techniques to help estimate test costs. Test costs that should include in the plan are:

- ❖ Costs of planning and designing the tests
- ❖ Costs of acquiring the hardware and software necessary for the tests
- ❖ Costs to support the test environment
- ❖ Costs of executing the tests

- ❖ Costs of recording and analyzing test results
- ❖ Tear down costs to restore the environment

## 21. Risks and Contingencies

Every testing effort has risks associated with it. Testing software with a high degree of criticality, complexity, or a tight delivery deadline all impose risks that may have negative impacts on project goals.

These risks should be:

- (i) Identified, (ii) Evaluated in terms of their probability of occurrence, (iii) Prioritized, and (iv) Contingency plans should be developed that can be activated if the risk occurs.

### Review Questions

1. Which of the following is not part of the test plan?
 

|                                    |                    |
|------------------------------------|--------------------|
| A. Schedule                        | C. Test Strategy   |
| B. Requirement Traceability Matrix | D. Incident Report |
2. What is the key objective of the test plan, List and explain at least five important components of the test plan document
3. You are part of a testing team working on a new online banking system that includes a feature for transferring funds between accounts. One of the risks identified during the risk assessment is that the system may be vulnerable to hacking or other security breaches. How would you manage this risk during testing?
4. Which of the following are the typical tasks that should be performed by the test leader
  - i) Prepare and acquire a test plan
  - ii) Write test summary reports based on the information gathered during testing.
  - iii) Database administration
  - iv) Balance the test resources accordingly
  - v) Provide guidelines on how to manage defects

|             |                    |
|-------------|--------------------|
| A. ii and v | C. i, iii and v    |
| B. i and v  | D. ii, iii, and iv |

## Chapter Five: Quality Management System and Standards

### Objectives

*At the end of this chapter students will be able to*

- ☞ understand quality management processes
- ☞ The awareness of software quality and the standards of practice
- ☞ Apply software quality knowledge and computing standards, software metrics and bench marks to produce innovative designs of software systems and components.
- ☞ understand the importance of standards in the quality management process and their impact on the final product

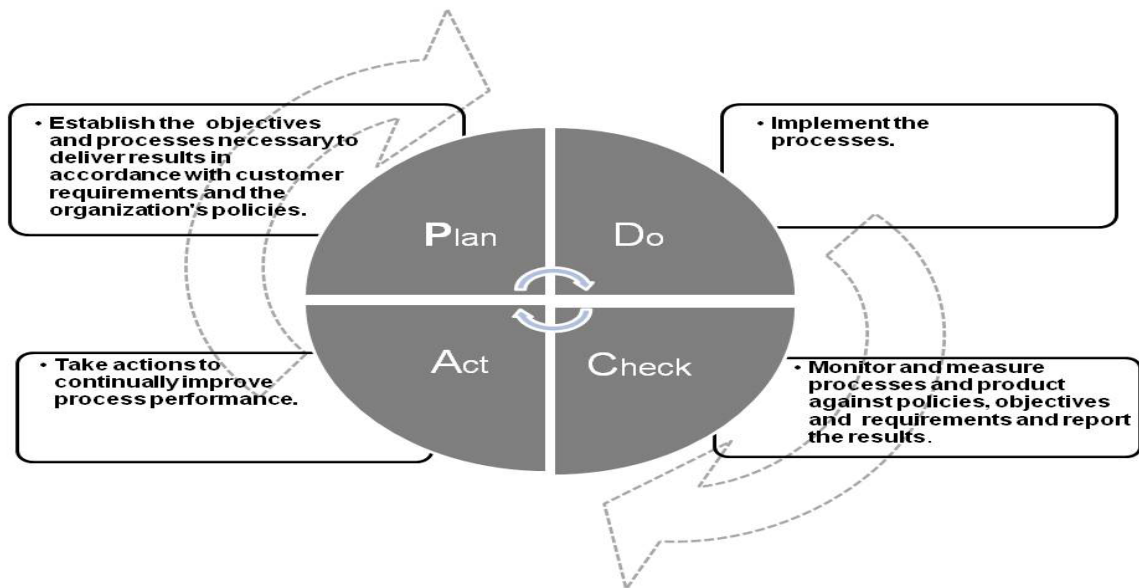
### 5.1 Quality Management System

A quality management system is a set of policies, processes, and procedures that establish a framework for organizations to follow in order to meet the requirements of their customers, regulatory bodies, and other stakeholders. It is designed to help organizations ensure that they consistently produce quality products and services while meeting customer need. Concerned with ensuring that the required level of quality is achieved in a software product. Involves defining appropriate quality standards and procedures and ensuring that these are followed. Should aim to develop a 'quality culture where quality is seen as everyone's responsibility.

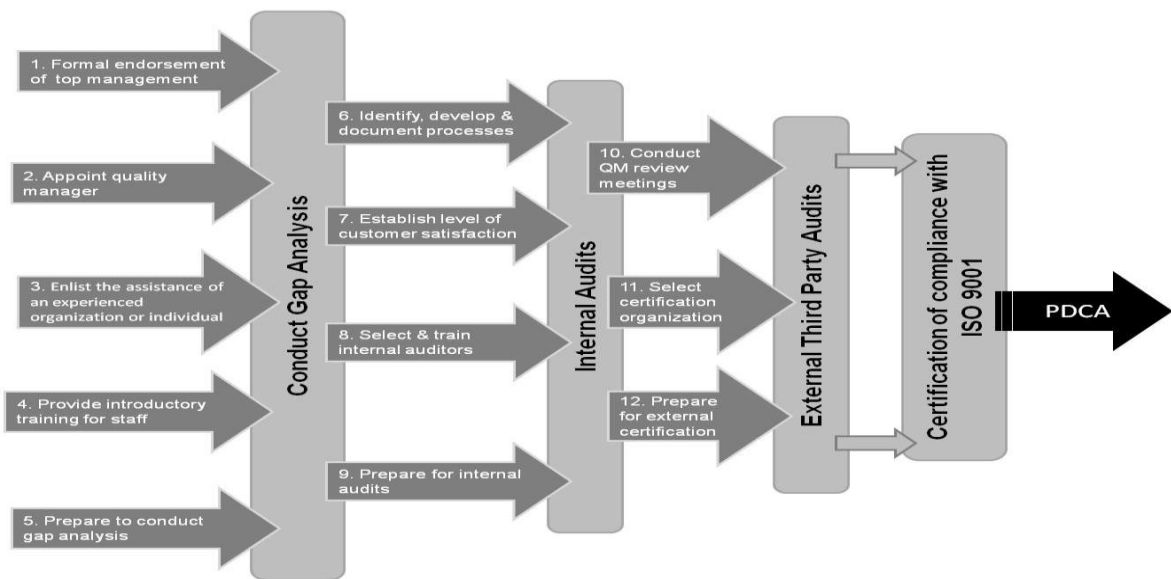
#### 5.1.1 QMS: Plan- Do- Check- Act

PDCA (plan–do–check–act) cycle is an iterative four-step management process typically used in QM orientated organizations.

- ❖ **Step 1** – Plan the processes, products, or services for your project.
- ❖ **Step 2** – Do the work to execute your project in accordance with the plan
- ❖ **Step 3** – Check and measure to assure that activities within the project meet the desired requirements and accomplish the project goals.
- ❖ **Step 4** – Act to continually improve all processes and strive in improving the delivery of the project.



### Steps for Implementing A Quality Management System



#### 5.1.2 Quality Standards

Quality-orientated process approaches and standards are maturing and gaining acceptance in many companies. Standards emphasize communication and shared understanding. For example: if one person says, “*Testing is complete*”, will all affected bodies understand what those words mean? This kind of understanding is not only important in a global development environment; even a small group working in the same office might have difficulties in communication and understanding of shared issues. Standards can help in these and other areas to make the business more profitable because less time is spent on non-productive work

## **Benefit of Standards**

The ability to apply software development and maintenance methodologies and procedures of the highest professional level. Better mutual understanding and coordination among development teams but especially between development and maintenance teams. Greater cooperation between the software developer and external participants in the project. Better understanding and cooperation between suppliers and customers, based on the adoption of known development and maintenance standards as part of the contract.

### **5.1.3 Organizations for Standard**

The following institutes and organizations, among the most prominent developers of SQA and software engineering standards, have gained an international reputation and standing in this area:

- ❖ IEEE (Institute of Electrical and Electronics Engineers) Computer Society
- ❖ ISO (International Organization for Standardization)
- ❖ DOD (US Department of Defense)
- ❖ ANSI (American National Standards Institute)
- ❖ IEC (International Electrotechnical Commission)
- ❖ EIA (Electronic Industries Association).

#### **1. ISO 9000 Standards**

The ISO 9000 standards developed by the International Standards Organization are both concerned with quality and process management. The specific ISO standard of concern to software organizations is ISO 9001. Common Questions

- ❖ At what CMM level is an ISO compliant organization?
- ❖ Can a Level 2 or (3) organization be considered ISO compliant?
- ❖ Should SPI be based on CMM or ISO?

The ISO series of standards is a set of documents dealing with quality systems that can be used for external quality assurance purposes. They specify quality system requirements for use where a contract between two parties requires the demonstration of the supplier's capability to design and supply a product. The two parties could be an external client and a supplier, or they could both be internal.

ISO 9000 is a guideline that clarifies the distinctions and interrelationships between quality concepts and provides guidelines for the selection and use of a series of international standards on quality systems that can be used for internal quality management purposes (ISO 9004) and for external quality purposes (ISO 9001, 9002, and 9003).

The quality concepts addressed by these standards are:

- ❖ An organization should achieve and sustain the quality of the product or service produced to continually meet the purchaser's stated or implied needs
- ❖ An organization should provide confidence to its own management that the intended quality is achieved
- ❖ An organization should provide confidence to the purchaser that the intended quality is being achieved in the delivered product or service provided

ISO 9001, "Quality Systems Model for quality assurance in development, production, installation, and servicing," is the ISO standard that applies to software development and maintenance. There is a guideline, ISO 9000-3, for applying ISO 9001 to software processes. A British guide for 9001 provides additional guidelines on using ISO 9000-3 and 9001 in the software area.

### **Contrasting ISO 9001 and CMM**

Some issues in ISO 9001 are not covered in CMM, and vice versa. The levels of detail differ. Chapter 4 in ISO 9001 is 5 pages long, sections 5, 6, and 7 in ISO 9000-3 comprise 11 pages; CMM is over 500 pages long.

The ISO 9001 clauses with no strong relationship to CMM KPAs are control of customer-supplied products and handling, packaging, preservation and delivery. The clause in ISO 9001 that addresses servicing in CMM in a completely distributed fashion is servicing. There is significant debate about the exact relationships to CMM for corrective and preventive action and statistical techniques.

The biggest difference is the emphasis in CMM on continuous process improvement. ISO only addresses minimum criteria for an acceptable quality system. CMM focuses strictly on software, while ISO 9001 has included hardware, software, processed materials, and services.

For both CMM and ISO 9001, the bottom line is “Say what you do; do what you say.” Every Level 2 KPA is strongly related to ISO 9001 Every KPA is at least weakly related to ISO 9001.

A CMM Level-1 organization can be ISO 9001 certified; that organization would have significant Level-2 process strengths and noticeable Level-3 strengths. Given a reasonable implementation of the software process, a ISO 9001 certified organization should be at least close to CMM Level-2.

Can a CMM Level-3 organization be considered ISO 9001 compliant? Even a Level-3 organization would need to ensure that delivery and installation are addressed, but even a Level-2 organization would have comparatively little difficulty in obtaining ISO 9001 certification.

### **How ISO 9001 Fits into the Software**

World (F. Coallier, *IEEE Software*, January 1994)

ISO 9001 has a strong emphasis on traditional manufacturing quality control. It assumes products are purchased in a formal, contractual environment with detailed specifications that are correct. Such an environment is not the case for consumer or mass-market products, however, and it is naive to assume such conditions for complex products like those that incorporate software.

### **CMM/ISO 9001 -1**

Software products are inherently complex and challenging to scope, develop, implement, verify, validate, and maintain. This requires a total-quality approach focused on customer satisfaction and continuous improvement. In ISO 9001, continuous improvement is almost totally absent. It merely addresses the control of a nonconforming product and recommends corrective and preventive action. For an organization that develops and manufactures embedded software products, an ISO 9001 certification tells very little about its software development capability. Certification means only that some basic practices are in place.

CMM is a more comprehensive model to measure software development capability. It covers more processes and has a five-level rating system that emphasizes continuous improvement. With ISO 9001, once you are certified, your challenge is only to maintain certification.

CMM can be used as a self-assessment. ISO 9001 certification requires auditors, which places emphasis on opinions of outsiders whose abilities may be unknown or marginal. ISO certification is usually prompted because certification is needed to get contracts. CMM review is usually done to improve and involves a more detailed study than does an ISO review.

ISO 9001 can still be worthwhile if: The auditors are good

- ❖ If the organization is CMM Level 1 or 2 because ISO 9001 covers the basics and can help the Organization grow.

## 2. Six Sigma

Six Sigma is one of the most popular quality methods lately. It is the rating that signifies “best in class,” with only 3.4 defects per million units or operations (DPMO). Its concept works and results in remarkable and tangible quality improvements when implemented wisely. Today, Six Sigma processes are being executed in a vast array of organization and in a wide variety of functions. Fueled by its success at large companies such as Motorola, General electric, Sony, and Allied Signal, the methodology is proving to be much than just a quality initiative.

The six-sigma framework of Define, Measure, Analyze, Improve, Control (DMAIC) is used to improve and optimize existing processes and products

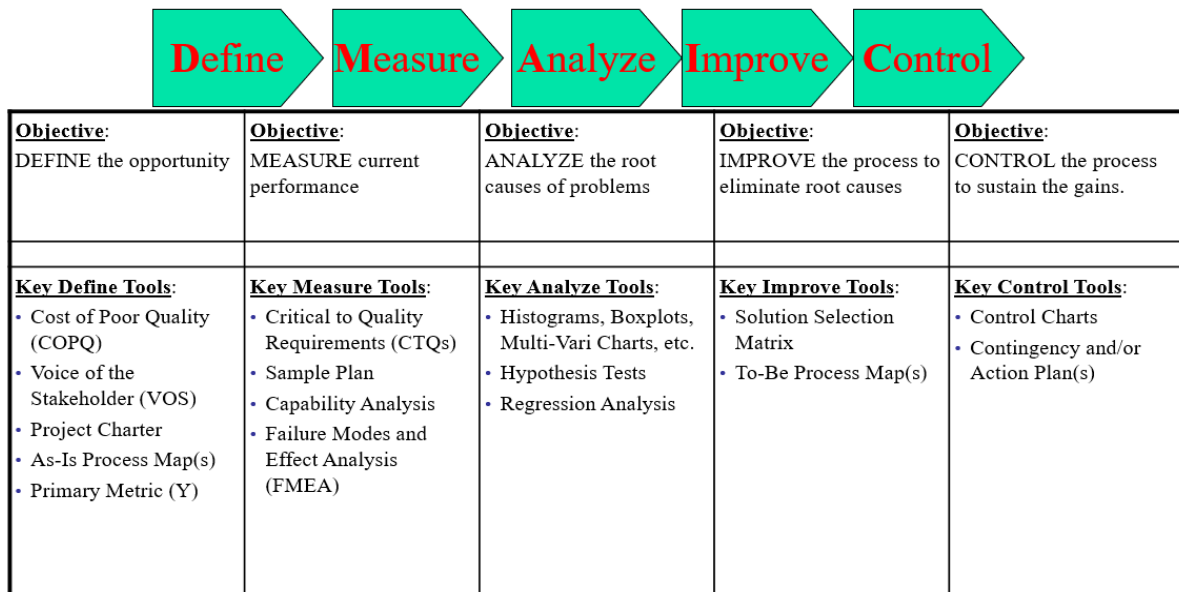


Figure 11: DMAIC The Improvement Methodology

The goal of Six Sigma is not to achieve six sigma levels of quality, but to improve profitability. Prior to Six Sigma, improvements brought about by quality programs, such as Total Quality Management (TQM) and ISO 9000, usually had no visible impact on a company's net income. In general, the consequences of immeasurable improvement and invisible impact caused these quality programs gradually to be. Six Sigma was originally developed as a set of practices designed to improve manufacturing processes and eliminate defects, but its application was subsequently extended to other types of business processes as well. In Six Sigma, a defect is defined as anything that leads to customer dissatisfaction.

### **Important of Six Sigma Standards**

- ❖ Six Sigma stands for six standard deviation from mean (sigma is the Greek letter used to represent standard deviation in statistics).
- ❖ Six Sigma methodologies provide the techniques and tools to improve the capability and reduce the defects in any process.
- ❖ Six Sigma improves the process performance, decreases variation and maintains consistent quality of the process output. This leads to defect reduction and improvements in profits, product quality and customer satisfaction.
- ❖ Six Sigma incorporates the basic principles and techniques used in business, statistics and engineering.
- ❖ The objective of Six Sigma principle is to achieve zero defects products/ process. It allows and engineering.
- ❖ The objective of Six Sigma principle is to achieve zero defects products/ process. It allows 4.4 defects per million opportunities.

### **Features of Six Sigma**

Apart from previous quality improvement initiatives include—

- ❖ A clear focus on achieving measurable and quantifiable financial returns from any Six Sigma project.
- ❖ An increased emphasis on strong and passionate management leadership and support.
- ❖ A special infrastructure of “Champions, “Master black Belts, “black Belts, “etc. to lead and implement the Six Sigma approach.
- ❖ A clear commitment to making decisions on the basis of verifiable data, rather than assumptions and guesswork.

## Review Questions

1. What are the advantages of standards, as well as list and explain software quality standards?
2. Why are these large companies embracing Six Sigma? What makes this methodology different from the others?
3. A software development company has been experiencing a high rate of defects in their product, resulting in frequent customer complaints and a significant amount of rework. You have been brought in as a Six Sigma consultant to lead a quality improvement project. Using the DMAIC methodology, outline the steps you would take to identify and eliminate the root causes of defects in the software product, to improve the overall quality of the product.

## Bibliography

- [1] D. Galin, Software Quality Assurance From Theory to Implementation, Great Britain by Biddles Ltd, Guildford and King's Lynn, 2004.
- [2] Anuja Arora and Madhavi Sinha, "Dynamic Content Testing of Web Application Using User Session Based State Testing," in *The Next Generation Information Technology Summit (4th International Conference)*, Noida, India, 2013.
- [3] Xinyu Han, Nan Zhang, Wei He, Kai Zhang and Longli Tang, "Automated Warship Software Testing System Based on LoadRunner Automation API," in *IEEE International Conference on Software Quality, Reliability and Security Companion*, Lisbon, Portugal, July 2018.
- [4] R. Abbas, Z. Sultan, and S. N. Bhatti, "Comparative Study of Load Testing Tools: Apache JMeter, HP LoadRunner," *Microsoft Visual Studio (TFS), Siege. Sukkur IBA Journal of Computing and Mathematical Sciences*, p. 2, Dec 2017.
- [5] Milad Hanna, Amal Elsayed and Mostafa-Sami, "Automated Software Testing Frameworks," *International Journal of Computer Applications (0975 – 8887)*, vol. 179 – No.46, June 2018.
- [6] Rakesh Kumar, Utkalika Satapathy and Meenu Dey, "Comparative Analysis on Automated Testing of Web-based Application," in *International Conference on Advances in Computing, Communication Control and Networking (ICACCCN2018)*, Bhubaneswar, 2018.
- [7] Tom Wissink and Carlos Amaro, "Successful Test Automation for Software Maintenance(ICSM)," in *Proceedings of the 22nd IEEE International Conference on Software Maintenance*, Philadelphia, PA, USA, 2016.
- [8] Muhammad Iqbal, and Uzair khan, "An Automated Model-based Approach to Repair Test Suites of Evolving Web," *The journal of Systems and Software*, 2020.

- [9] T. Admin, "Explore the Scripting Techniques Used in Test Automation," Digital Thoughts , 08 Jun 2021. [Online]. Available: <https://blog.thedigitalgroup.com/explore-the-scripting-techniques-used-in-test-automation>. [Accessed 10 Jun 2021].
- [10] J. Kent, "Test Automation From RecordPlayback to Frameworks," *Simply Software Testing Technology* , 2019.
- [11] Milad Hanna,Nahla and Mostafa , "A Review of Scripting Techniques Used in Automated Software Testing," *International Journal of Advanced Computer Science and Applications*, vol. 5, 2014.
- [12] Ranjit Shewale and Sanjeev Kulkarni, "A Review of Scripting Techniques Used in Automated Software Testing," linkedin, February , 2016.
- [13] K. V. Arya and H. Verma, "Keyword Driven Automated Testing Framework for Web Application," in *2018 9th International Conference on Industrial and Information Systems (ICIIS)*, Gwalior, India, 2018.
- [14] Giuseppe A. Di Lucca, Anna Rita Fasolino, "Web Application Testing," 2018.